PRESENTER NOTES · SHOW AND TELL

The Next Word Is Ours

LLMs and the systems around them: how language models work, how AI helps us, and the harness we have built around a model. Sixteen slides, about fifteen minutes, then questions.

How to use these notes. Each slide has its own page: a screenshot, what the audience sees, what to say, what to click, the facts to have ready, the line into the next slide and the questions it may raise. After the slides come grouped questions and answers, the full glossary, backup material from slides that were cut, and the sources behind every figure.

Every figure on the slides marked “illustrative” is an example, not a measurement. Public figures are cited in Appendix C.

BEFORE YOU PRESENT

Set-up checklist

1. Open the deck's **index.html** (the presentation folder on the main branch) in Chrome or Edge, full screen (F11). Keep browser zoom at 100%. The deck is built for a PC screen; at 1,440 px wide and above the bottom bar also shows the slide's name.
2. Be online if you can: the fonts come from Google Fonts. Offline, it falls back to system fonts and still fits.
3. Turn off the operating system's "reduce motion" setting, or the animations and auto-plays are skipped (everything is still shown, just without motion).
4. Rehearse the interactive moments once: the training demo (slide 3), the dials (slide 6), Come back tomorrow (slide 7), the salary question and Change §4.1 (slide 12), Restart billing → Approve as admin (slide 13), Approve/Edit/Reject and Restart the server (slide 14), the failed run (slide 15).
5. Press **Start over** on the training demo (slide 3) before the talk; it keeps its state between visits.
6. Confirm which parts of the proof of concept run today, so you can answer "is this real?" precisely.

Controls

→ ↓ Page Down Space, scroll	Next slide (Space presses a focused button instead)
← ↑ Page Up	Previous slide (on a focused slider, arrows move the slider)
Home / End	First / last slide (on a focused slider: its ends)
G, or click the counter	Slide list grouped by part; click or Enter to jump; Esc closes
‹ › buttons	Previous and next slide, at the bottom right of the bar
Engineer's note button	In the bottom bar, beside the ‹ › buttons, on slides with a technical deep-dive (2, 3, 5, 6, 7, 11, 12, 13, 14, 15); Esc closes it
Bottom bar	The three parts; hover a part to see its name, click to jump to its start
Last slide	Revisit part 1/2/3 links, and Restart (back to slide 1)

RUNNING ORDER

Sixteen slides in three parts

Part 1, What an LLM is (slides 1–9) · **Part 2, The benefits of AI** (slide 10) · **Part 3, What we have built** (slides 11–15) · then the summary and questions (16). The chapter number in each slide's small heading (01–13) differs from the slide counter, because the title and the dark "frozen" slide are unnumbered.

#	SLIDE	TIME	INTERACTIVE
1	Welcome	0:40	Yes
2	What is an LLM	1:00	–
3	How it learns	1:20	Yes
4	The part most people miss	0:40	–
5	How it answers	1:40	Yes
6	Temperature and top-K	1:10	Yes
7	Memory	1:00	Yes
8	The vocabulary	0:20	Yes
9	Myths	0:45	Yes
10	How AI helps us	0:50	Yes
11	Our harness	0:50	Yes
12	Our documents: RAG	1:30	Yes
13	Multi-agent orchestration	1:30	Yes
14	Human in the loop	1:00	Yes
15	Observability	1:00	Yes
16	Summary and questions	0:30 + Q&A	Yes

Total about 16 minutes. To save time: skim the glossary (8) and myths (9), and on slide 5 jump straight to steps 2, 3 and 7.

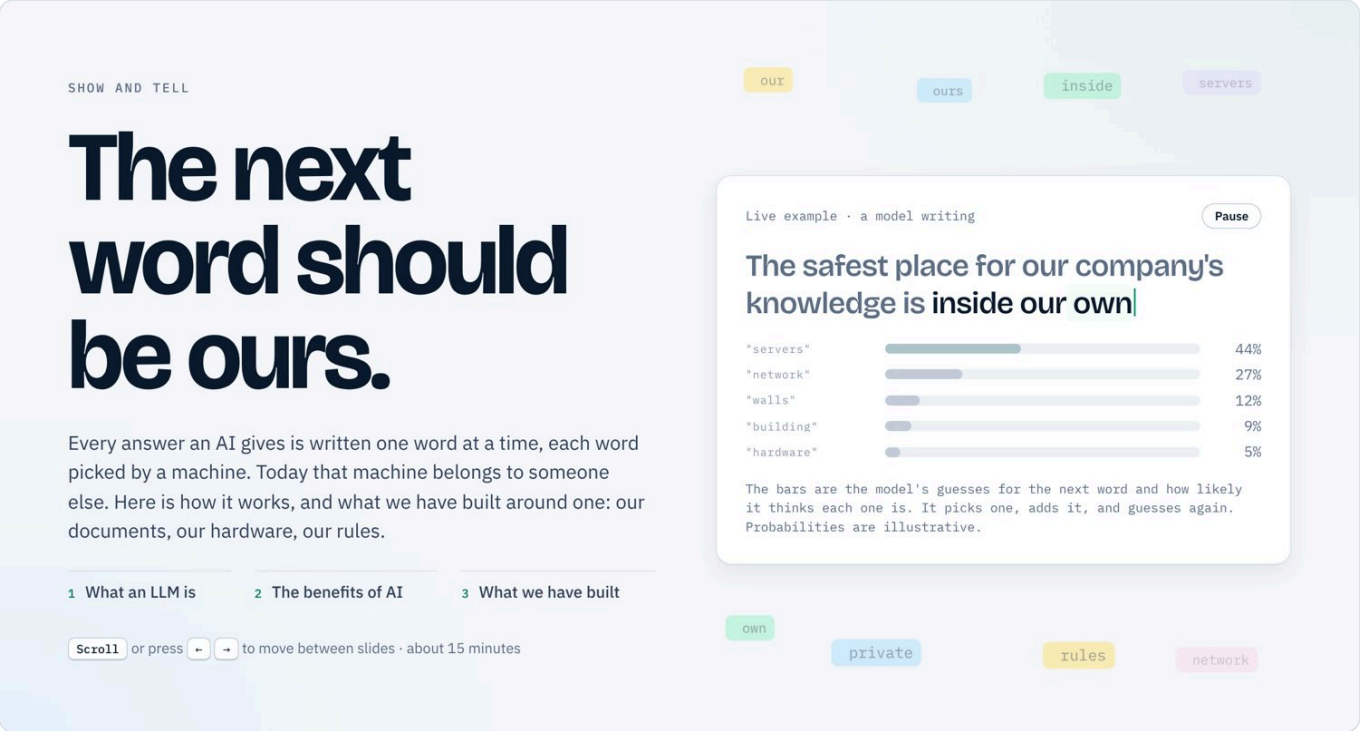
THE STORY IN ONE PARAGRAPH

An LLM predicts the next word (1–2), learned by guessing trillions of times (3), and is frozen afterwards, so it invents what it does not know (4). Inside, text becomes tokens and numbers, attention and layers refine them, and one token is picked at a time (5), with temperature and top-K setting how daring the picks are (6). It has no memory of its own; the harness re-sends the conversation (7). After the vocabulary and the myths (8–9), the evidence that AI speeds up real work (10). Then what we built: the harness (11) and each of its rings: our documents through RAG with access rights (12), tools through a LangGraph supervisor and three agents (13), rules through human approval (14) and the audit log through LangSmith traces (15). The close brings back the opening line (16).

SLIDE 1

Welcome

Chapter Title · suggested time 0:40



Purpose. Set up the idea that runs through the whole talk: every AI answer is written one predicted word at a time, and the machine choosing those words could be ours.

ON SCREEN

- The headline **“The next word should be ours.”** writes itself in highlighter colours.
- A **live example** card on the right: a model completing “The safest place for our company’s knowledge is inside our own …”, with bars for its candidate next words (servers 44%, network 27%, walls 12%, building 9%, hardware 5%). It picks one, adds it and guesses again, on a loop.
- The paragraph beneath it: “Every answer an AI gives is written one word at a time, each word picked by a machine. Today that machine belongs to someone else. Here is how it works, and what we have built around one: our documents, our hardware, our rules.”
- The agenda: **1 What an LLM is · 2 The benefits of AI · 3 What we have built**, and the hint “about 15 minutes”.
- The small heading reads **Show and Tell**.

WHAT TO SAY

- “Every answer an AI gives you is written one word at a time. At each step a machine looks at everything so far and picks the next word. Today, that machine belongs to someone else.”
- Point at the live card: “Those bars are the model’s guesses for the next word, with how likely it thinks each is. It picks one, adds it, and guesses again. That loop is all an LLM does, and the rest of this talk builds on it.”
- Walk through the agenda in one breath: what an LLM is and how it works; where AI helps us; and what we have already built around a model.
- Optional framing line: “Today is a show and tell: what these systems are, and the proof of concept we have built.”

WHAT TO CLICK

1. The live card plays on its own. **Pause** (top right of the card) freezes it if you want to talk over one frame.

2. The three agenda items are links: clicking one jumps to the start of that part (useful in Q&A).
3. Advance with → / Page Down / Space / the scroll wheel.

FACTS TO HAVE READY

- The probabilities in the live card are illustrative, and the card says so.

Into the next slide. “So what exactly is the machine doing that guessing? Let’s start with what an LLM actually is.”

QUESTIONS THIS SLIDE MAY RAISE

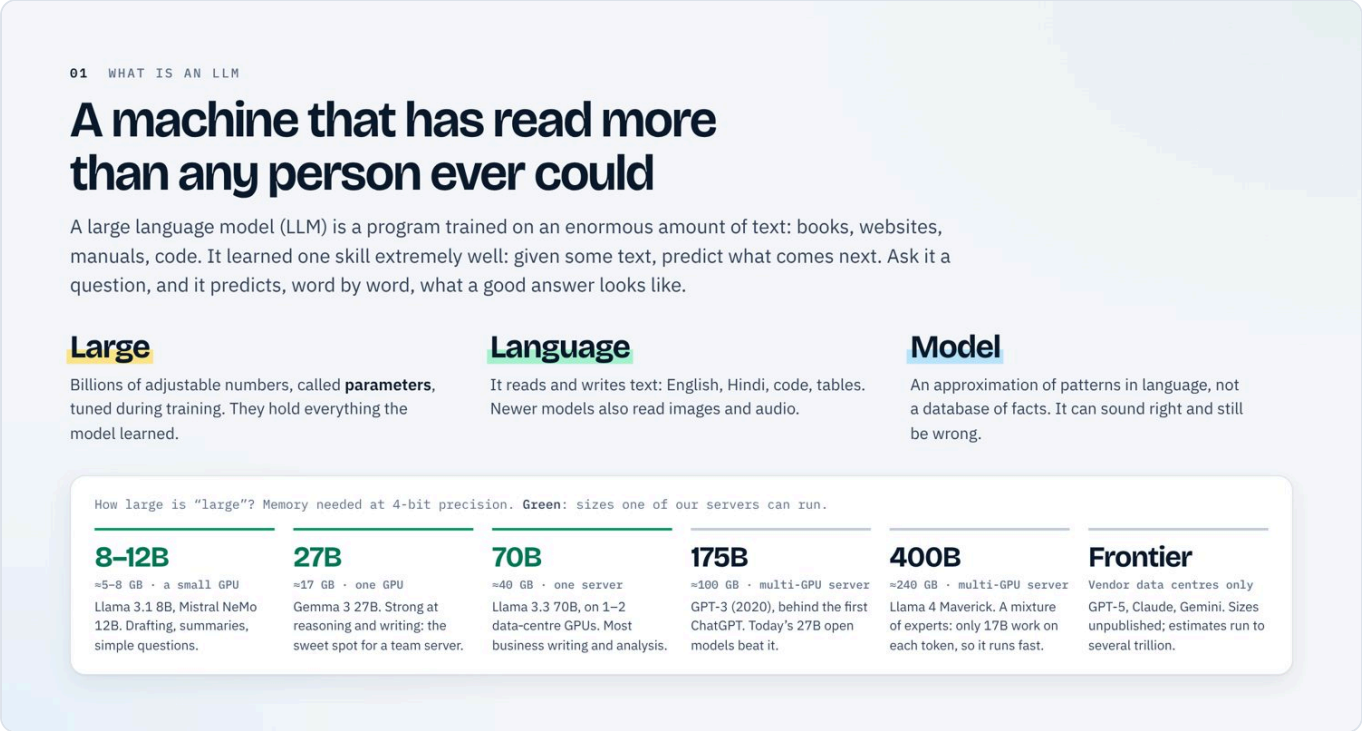
Is this a proposal?

> No, it is a show and tell: how these systems work and what we have built. If people ask what would come next, the usual answer is a small pilot with one department (see Appendix B for the earlier pilot outline).

SLIDE 2

What is an LLM

Chapter 01 · suggested time 1:00



Purpose. Define an LLM in plain words and give a feel for model sizes, so "8B", "70B" and "frontier" mean something for the rest of the talk.

ON SCREEN

- Headline: **"A machine that has read more than any person ever could."**
- Three facts: **Large** (billions of adjustable numbers called parameters), **Language** (reads and writes English, Hindi, code, tables; newer models also images and audio), **Model** (an approximation of patterns, not a database of facts: it can sound right and still be wrong).
- A **size ladder**, memory needed at 4-bit precision; green rungs are sizes one of our servers can run:
- 8–12B ≈ 5–8 GB, a small GPU (Llama 3.1 8B, Mistral NeMo 12B: drafting, summaries) · 27B ≈ 17 GB, one GPU (Gemma 3 27B: the sweet spot for a team server) · 70B ≈ 40 GB, one server (Llama 3.3 70B on 1–2 data-centre GPUs) · 175B ≈ 100 GB (GPT-3, 2020; today's 27B open models beat it) · 400B ≈ 240 GB (Llama 4 Maverick, a mixture of experts: only 17B work on each token) · Frontier: vendor data centres only (GPT-5, Claude, Gemini; sizes unpublished, estimates run to several trillion).

WHAT TO SAY

- "An LLM is a program trained on an enormous amount of text. It learned one skill extremely well: given some text, predict what comes next. Ask it a question and it predicts, word by word, what a good answer looks like."
- Stress the third fact: "It is a model of language, not a database. That is why it can sound confident and still be wrong. Hold that thought; it comes back in two slides."
- On the ladder: "Size is measured in parameters. The green ones are sizes a single server of ours can run. Notice GPT-3, the model behind the first ChatGPT: today's 27-billion-parameter open models beat it."
- On Maverick: "Mixture of experts means only a small part of the model works on each word, so a very large model can still run fast."

WHAT TO CLICK

1. No interaction. The ladder rungs rise in sequence and the green rungs glow once.
2. The Engineer’s note (bottom-right button) has the memory formula if anyone asks how the GB figures are worked out.

FACTS TO HAVE READY

- Memory for the weights ≈ parameters × bits per parameter ÷ 8. An 8B model: 16 GB at 16-bit, about 4.5–5 GB at 4-bit; a 70B model about 40 GB at 4-bit.
- The conversation itself (the KV cache) needs extra memory, growing with context length.
- Mixture-of-experts models must hold all weights in memory but only compute with the active ones.

Into the next slide. “How does a pile of random numbers become something that can write? By guessing, trillions of times.”

QUESTIONS THIS SLIDE MAY RAISE

What does “4-bit precision” mean?

> Each parameter is stored with 4 bits instead of 16 (this is called quantisation). The model needs about a quarter of the memory for a small loss in quality; 4-bit keeps most of the quality for everyday work.

Which of these would we actually use?

> The proof of concept uses gpt-oss-120b, gpt-oss-20b and Gemma 3 27B (slides 13–15). gpt-oss-120b is a mixture of experts like Maverick: 117B parameters in total but only about 5.1B work on each token, and it fits on one 80 GB GPU.

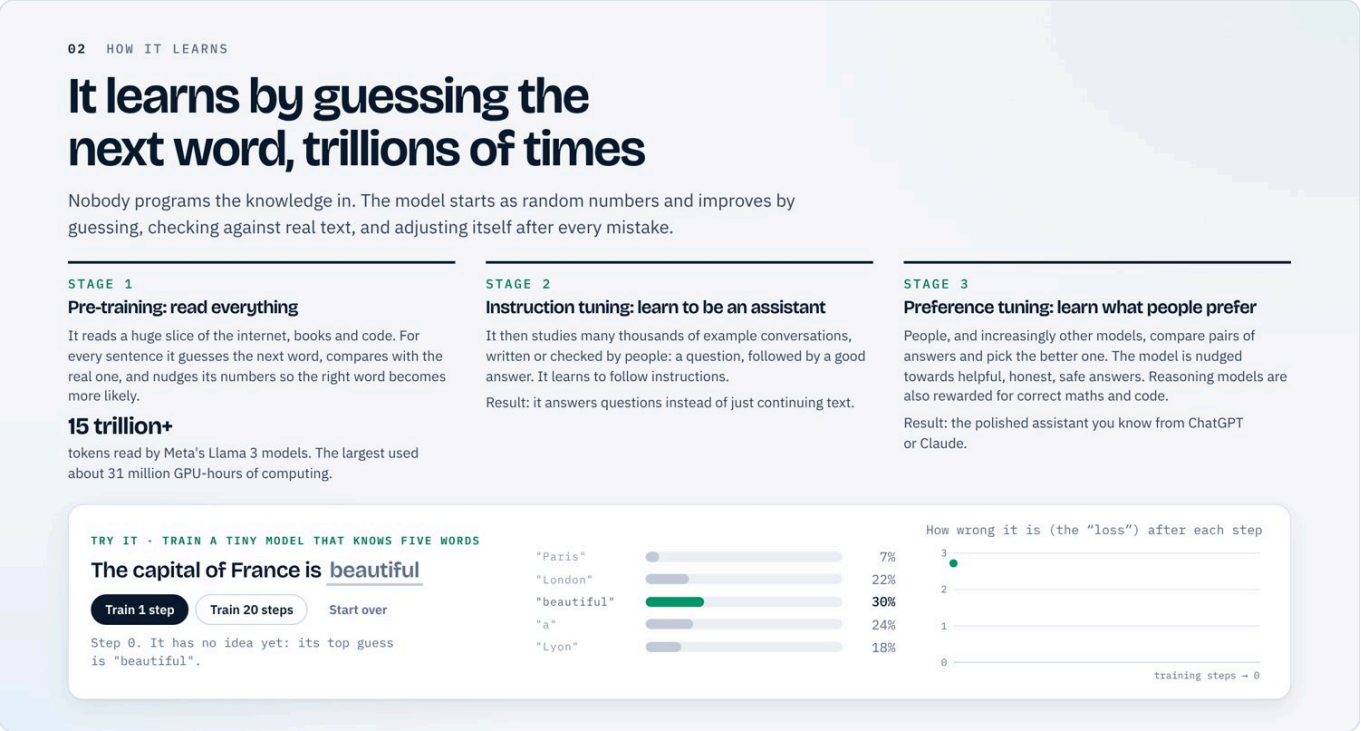
Is bigger always better?

> No. For questions about our own business, giving the model the right documents matters more than raw size (myth 8 on slide 9).

SLIDE 3

How it learns

Chapter 02 · suggested time 1:20



> **Purpose.** Show that nobody programs knowledge in: the model learns by guessing the next word and correcting itself. The demo makes this tangible.

ON SCREEN

- Headline: **“It learns by guessing the next word, trillions of times.”**
- Three stages: **1 Pre-training** (read a huge slice of the internet, books and code; guess the next word, compare, nudge; Llama 3 read 15 trillion+ tokens and the largest used about 31 million GPU-hours) · **2 Instruction tuning** (study thousands of example conversations; it learns to follow instructions) · **3 Preference tuning** (people, and increasingly other models, pick the better of two answers; reasoning models are rewarded for correct maths and code).
- A **Try it** card: a tiny model that knows five words completes “The capital of France is ____”, with bars for Paris, London, beautiful, a and Lyon, a loss chart, and three buttons.

WHAT TO SAY

- “Nobody types the knowledge in. The model starts as random numbers. It reads a sentence, guesses the next word, checks against the real text, and nudges its numbers so the right word becomes a little more likely. Then it does that trillions of times.”
- Run the demo (see below) while you narrate: “Here is a model that knows five words. At the start it has no idea: its best guess is ‘beautiful’. Each training step shows it the right answer and nudges it closer…”
- “Real training does exactly this, for every word of every sentence, over trillions of words, adjusting billions of numbers at once. That is stage one. Stages two and three turn a text-continuer into an assistant that answers questions politely.”

WHAT TO CLICK

1. **Before you start**, press **Start over** if anyone has played with it (the demo keeps its state when you leave the slide).

2. Click **Train 1 step** twice: the blank still says “beautiful” (grey) and Paris climbs 7% → 12% → 22%.
3. Click once more (step 3): the blank turns green and says **Paris** (33%). “Now it has the right answer, but it is not sure yet.”
4. Click **Train 20 steps**: Paris climbs to about 93%, the loss curve falls towards zero, and the message reads “Learned”.
5. **Start over** resets it to step 0.

FACTS TO HAVE READY

- The demo does real gradient descent on five scores (logits): each step subtracts the learning rate (0.6) times the gradient of −log p(Paris).
- Llama 3: 15 trillion+ training tokens; Meta reports about 30.8 million H100 GPU-hours for Llama 3.1 405B.
- Stage 2 is supervised fine-tuning (SFT); stage 3 is RLHF or a direct method such as DPO; reasoning models add reinforcement learning with verifiable rewards.

Into the next slide. “And once training ends, something important happens…”

QUESTIONS THIS SLIDE MAY RAISE

What is the “loss”?

> A measure of how wrong the model is: here, how little probability it gives the right word. Training is the process of pushing the loss down.

How much does it cost to train a model like this?

> Frontier-scale training costs tens of millions of dollars or more (tens of crores of rupees) and needs thousands of GPUs for months. That is why we use models others have already trained and give them our documents instead (slides 4 and 12).

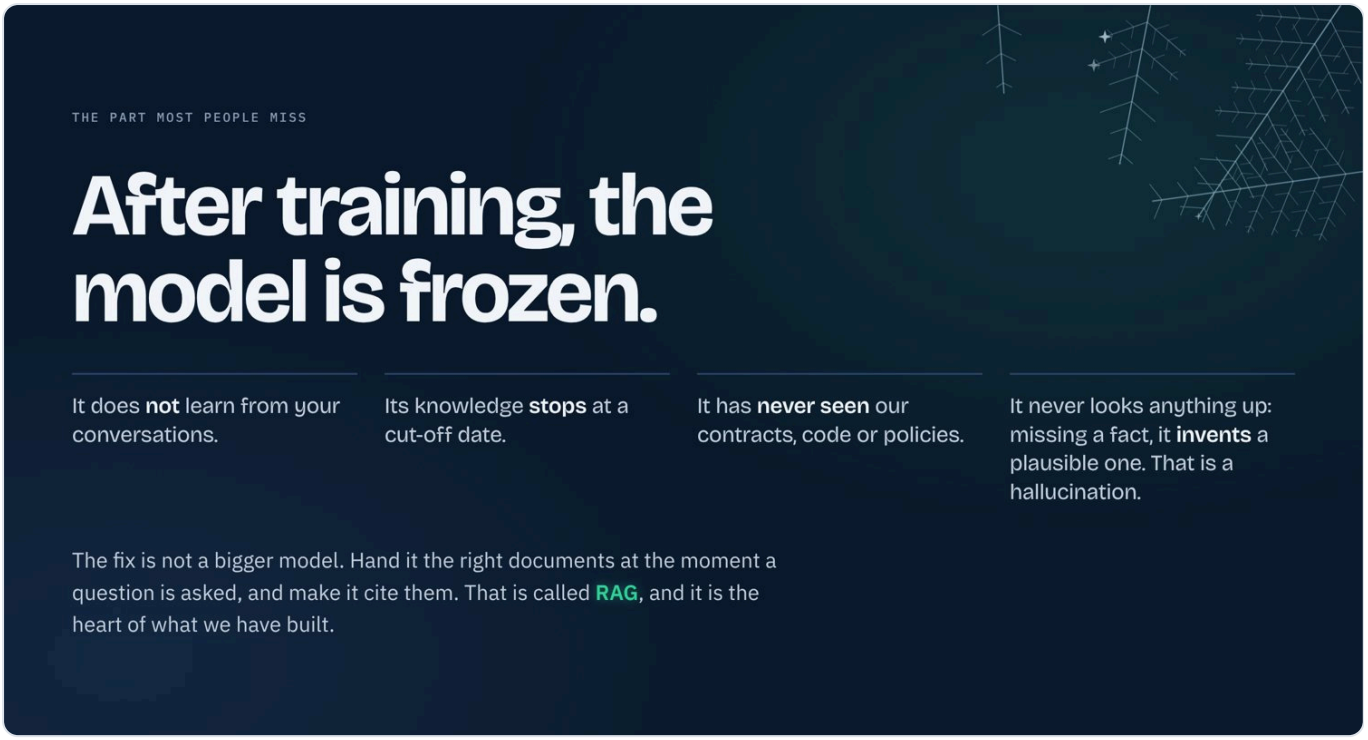
Can we train it on our data?

> We do not need to for knowledge: RAG hands it our documents at question time (slide 12). Fine-tuning (for example LoRA) is useful for style and format, not for reliably adding facts.

SLIDE 4

The part most people miss

Chapter (unnumbered, dark slide) · suggested time 0:40



Purpose. The single most important idea for a lay audience: a trained model is frozen, so it hallucinates on anything it has not seen. RAG is the fix.

ON SCREEN

- Dark slide. Headline: **“After training, the model is frozen.”** Frost forms in the corner.
- Four points: it does not learn from your conversations · its knowledge stops at a cut-off date · it has never seen our contracts, code or policies · it never looks anything up, so a missing fact becomes a plausible invention, a **hallucination**.
- Foot: “The fix is not a bigger model. Hand it the right documents at the moment a question is asked, and make it cite them. That is called **RAG**, and it is the heart of what we have built.”

WHAT TO SAY

- Slow down here. “This is the part most people miss. When training ends, the model is frozen. It doesn’t learn from your chats, it stops at a cut-off date, and it has never seen anything of ours.”
- “So what does it do when it doesn’t know? It doesn’t look anything up. It writes the most plausible words. That is a hallucination.”
- “The fix is not a bigger model. It is giving it the right documents at the moment you ask, and making it cite them. That is RAG, and we will see our version on slide 12.”

WHAT TO CLICK

1. No interaction.

Into the next slide. “Let’s open the machine up and see what happens in the second after you press Enter.”

QUESTIONS THIS SLIDE MAY RAISE

But ChatGPT remembers things about me?

> That is the app's separate memory feature, which re-sends saved text with each question (slide 7). The model itself is unchanged.

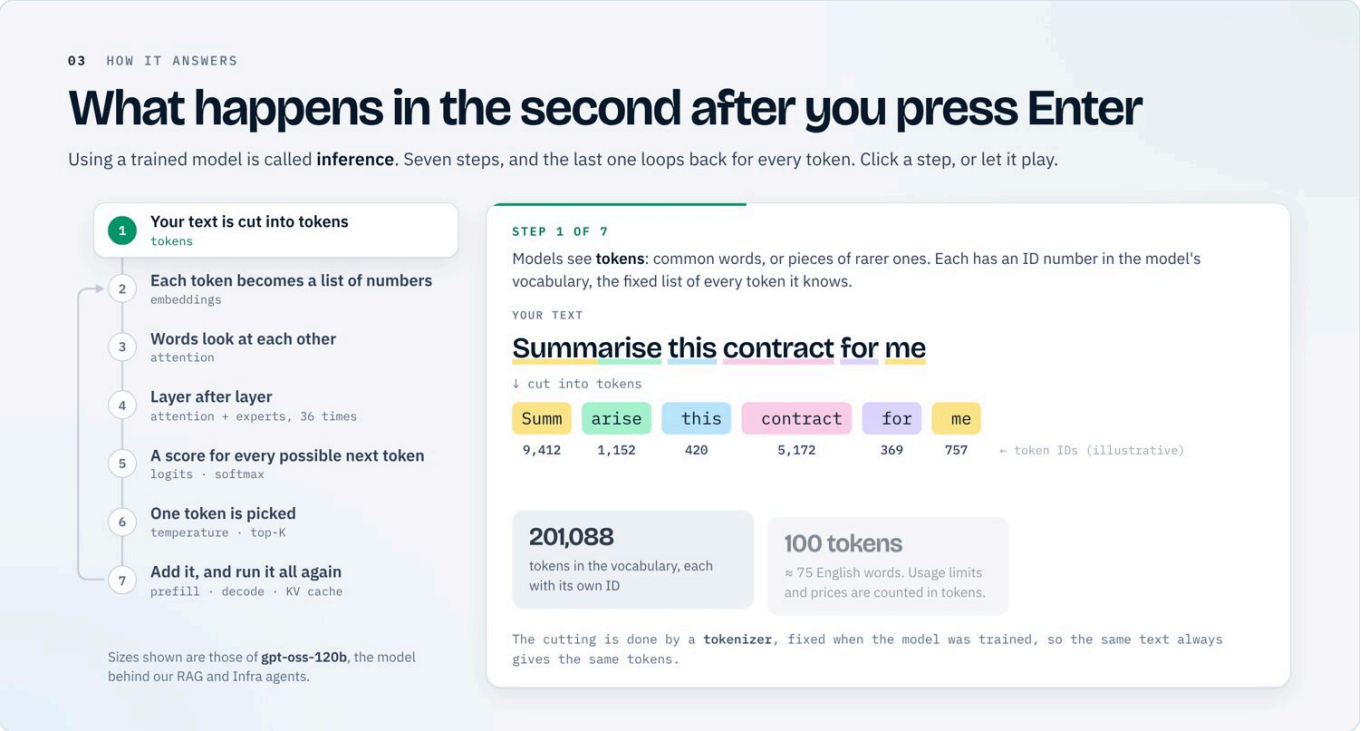
If it is frozen, how do models get updated?

> Vendors train new versions. With our own setup we swap in a newer open model when one is better; it is a settings change in the harness (slide 11).

SLIDE 5

How it answers

Chapter 03 · suggested time 1:40



Purpose. Inference, one level deeper: tokens, embeddings, attention, layers, scores, picking and the loop. Uses gpt-oss-120b's real configuration.

ON SCREEN

- Headline: **"What happens in the second after you press Enter."** A rail of seven steps on the left (with a loop arrow from 7 back to 2) and one animated panel on the right. Note under the rail: sizes are those of **gpt-oss-120b**, the model behind our RAG and Infra agents.
- **1 Tokens**: "Summarise this contract for me" becomes six tokens (Summ | arise | this | contract | for | me) with illustrative IDs. Vocabulary: 201,088 tokens. 100 tokens ≈ 75 English words; limits and prices are counted in tokens.
- **2 Embeddings** (the showpiece): the ID picks one row of a learned table (201,088 rows × 2,880 numbers). "contract" and "agreement" get almost the same numbers; "picnic" very different. A map of meaning (contract, agreement, lease close; picnic, banana far) with similarity scores (agreement 0.91, lease 0.84, invoice 0.62, picnic 0.07). Notes: position is tracked ("dog bites man" vs "man bites dog"); RAG uses a separate model that turns a whole passage into 384 numbers.
- **3 Attention**: each word asks a question (query), every word offers a label (key) and its meaning (value). Toggle between "I deposited cash at the bank" and "We had a picnic by the river bank"; lines show what "bank" attends to. Three example attention heads.
- **4 Layer after layer**: attention + feed-forward, 36 times. The feed-forward block is split into 128 **experts**; each token uses just 4. The "bank" vector sharpens towards money or river as it rises.
- **5 Scores**: raw scores (logits) → softmax → probabilities: This 38%, Here 27%, The 14%, Sure 11%, Summary 4%, and about 201,000 other tokens with tiny chances.
- **6 Picking**: temperature and top-K choose one (link to the next slide).

- **7 The loop**: the token joins the text and everything repeats. Prefill reads the prompt all at once; decode writes one token per trip; the KV cache keeps earlier work. Context window 131,072 tokens ≈ 98,000 words. Speed: people read ≈ 5 tokens/s; a good private setup writes 30 to 100+. A 500-word answer ≈ 650 trips through the whole model. gpt-oss-120b fits on one 80 GB GPU (expert weights at about 4.25 bits each).

WHAT TO SAY

- "Using a trained model is called inference. Seven steps, and the last one loops back for every single token of the answer."
- Step 1: "The model doesn't see words, it sees tokens: common words, or pieces of rarer ones. 'Summarise' becomes two."
- Step 2 (spend time here): "Each token's ID picks one row of a huge learned table: 2,880 numbers that describe its meaning. Words with similar meanings get similar numbers, so they sit close together on a map of meaning. That is an embedding, and the same idea powers our document search later."
- Step 3: "Then every word looks at every earlier word for context. Is this 'bank' about money or a river? Toggle the sentence and watch where 'bank' looks."
- Step 4: "That happens 36 times, layer after layer. In this model each layer's knowledge is split into 128 experts and each word only consults 4, which is why a 117-billion-parameter model runs fast."
- Steps 5–7: "At the end it scores every token in its vocabulary, picks one, adds it, and runs the whole thing again. A 500-word answer is about 650 trips through the model. That is why answers stream in word by word."

WHAT TO CLICK

1. On arrival the slide **plays itself**, about a minute in total (dwell per step: 6.5, 10.5, 8.5, 8.5, 7, 8, 11 s). A thin progress line runs along the top of the panel.
2. **Click any step** to take over; auto-play stops and you drive it.
3. In step 3, click the two sentence buttons to toggle "bank".
4. In step 6, the "Try both dials · next slide" link jumps to slide 6.
5. Short on time? Click straight to steps 2, 3 and 7 and summarise the rest.

FACTS TO HAVE READY

- gpt-oss-120b (published configuration): o200k_harmony vocabulary of 201,088 tokens; model width 2,880; 36 decoder layers; 128K (131,072) context; 128 experts per layer, 4 per token; 117B parameters, about 5.1B active per token; 64 query heads share 8 key-value heads; alternating full and 128-token sliding-window attention; MXFP4 expert weights (≈4.25 bits); fits one 80 GB GPU.
- Embedding values, similarity scores, attention weights and token IDs on the slide are illustrative and labelled so.
- Prefill is compute-bound; decode is memory-bandwidth-bound (every step reads all active weights). Decode ceiling ≈ memory bandwidth ÷ bytes of active weights.

Into the next slide. "Step 6 picked one token, with a controlled amount of randomness. Let's play with the two dials that control it."

QUESTIONS THIS SLIDE MAY RAISE

Why 201,088 tokens and not every word?

> A vocabulary of word pieces covers any text, including names, code and other languages, with a manageable table. Common words get one token; rare ones are split.

Are the embeddings on this slide the same as the ones used for RAG?

> No. These live inside the model, one per token. RAG uses a separate, small embedding model (all-MiniLM-L6-v2) that turns a whole passage into one list of 384 numbers for search.

What is a “mixture of experts”?

> Each layer has many specialist sub-networks (128 here) and a router sends each token to a few (4). The model holds all of them in memory but only computes with the active ones, so it runs at the speed of a much smaller model.

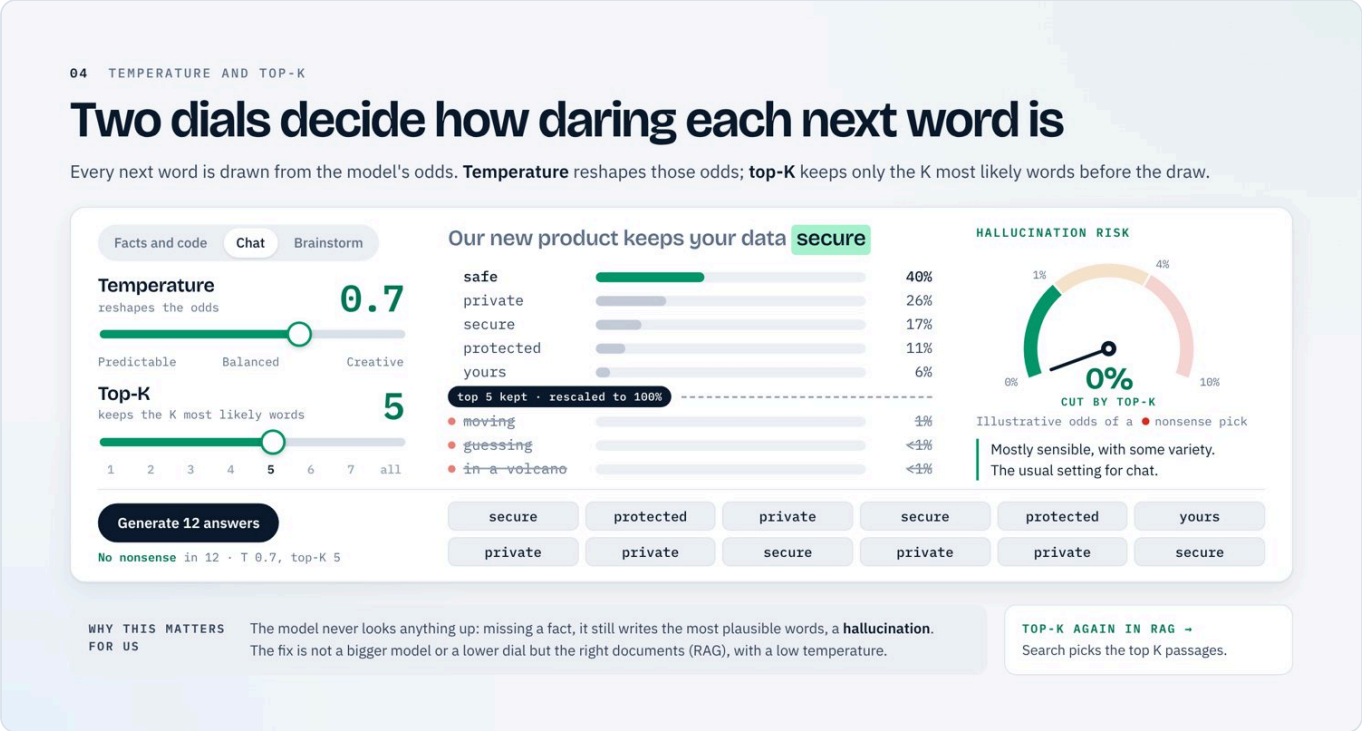
Why does it slow down with long documents?

> Everything must fit in the context window, and the KV cache grows with the length. Reading (prefill) is fast and parallel; writing (decode) is one token at a time.

SLIDE 6

Temperature and top-K

Chapter 04 · suggested time 1:10



Purpose. Show how two sampling dials trade predictability against creativity, and why low temperature plus the right documents is how we keep answers factual.

ON SCREEN

- Headline: **“Two dials decide how daring each next word is.”** Prompt: “Our new product keeps your data ___”.
- Left: presets **Facts and code · Chat · Brainstorm**, a **Temperature** slider (0.1–1.0, labelled Predictable · Balanced · Creative) and a **Top-K** slider (1–7 and all).
- Centre: live odds for eight words (safe, private, secure, protected, yours, and the nonsense words moving, guessing, in a volcano). A cut line shows which words top-K removes; the kept ones are rescaled to 100%.
- Right: a **Hallucination risk** gauge (illustrative odds of a nonsense pick) and a one-line takeaway that always matches the setting.
- **Generate 12 answers** draws 12 samples and counts any nonsense.
- Bottom: why it matters for us (low temperature plus RAG), and a link to the RAG slide.

WHAT TO SAY

- “The model gives every candidate word a probability. Temperature reshapes those odds: low makes the favourite even more likely; high flattens everything, so unlikely words get a real chance.”
- “Top-K is a guard rail: only the K most likely words are allowed at all. Set it to 1 and you always get the top word.”
- Use the gauge: “At high temperature with no top-K, nonsense like ‘in a volcano’ starts creeping in. That is one route to hallucination.”
- “For facts, code and answers from our documents we run low temperature, 0 to 0.3. Chat sits around 0.7. And remember: no dial makes a model know a fact it was never given. That is what RAG is for.”

WHAT TO CLICK

1. On arrival it **demonstrates itself** (about 10 s): Facts (T 0.2, K 1) → 12 answers, then glides to Brainstorm (T 1.0, K all) → 12 answers, then to Chat (T 0.7, K 5) → 12 answers.
2. **Touching any control stops the demo**, and it will not replay on that slide visit. That is intended.
3. Click presets, or drag the sliders (arrow keys on a focused slider adjust it without changing slide; Home/End go to its ends).
4. Click **Generate 12 answers** after each change; if you move a dial afterwards the old answers dim and say “Dials moved: generate again”.

FACTS TO HAVE READY

- Presets: Facts and code T 0.2, K 1 · Chat T 0.7, K 5 · Brainstorm T 1.0, K all.
- p = softmax(z / T). As T → 0 sampling becomes greedy (always the top token). At T = 1 the model’s own odds are used unchanged.
- The slide’s temperature dial stops at 1.0, the model’s natural odds. Many APIs allow up to 2, but above 1 answers quickly lose coherence.
- Top-p (nucleus) and min-p are alternatives to top-K; the order of temperature and top-K does not matter, but for top-p and min-p it does.
- Typical settings: chat ≈ 0.7 (often with top-p 0.9–0.95); extraction, code and RAG answers 0–0.3.
- The word scores and the gauge are illustrative.

Into the next slide. “So the model picks words well, but does it remember what you said a minute ago? Not on its own.”

QUESTIONS THIS SLIDE MAY RAISE

Why not always use temperature 0?

> For facts we nearly do. For drafting or brainstorming, a little randomness gives more natural, varied writing. Zero can also get repetitive.

Does lowering temperature stop hallucinations?

> It reduces random odd picks, but a model that lacks a fact will still confidently write a plausible one. The real fix is RAG with citations.

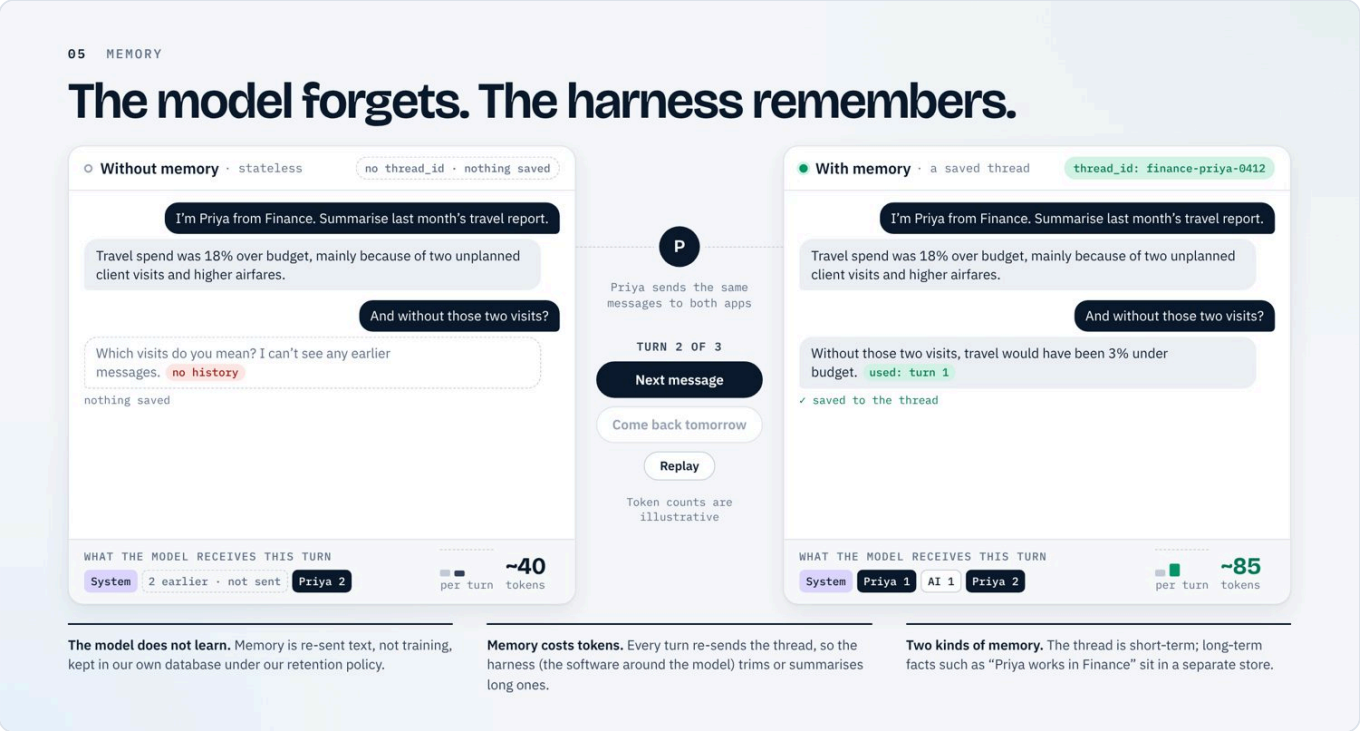
What is top-p?

> Instead of a fixed K, keep the smallest set of words whose probabilities add up to p (say 0.9). The shortlist shrinks when the model is confident and grows when it is not.

SLIDE 7

Memory

Chapter 05 · suggested time 1:00



Purpose. Show that the model itself is stateless, and memory is the harness re-sending the conversation (a thread) each turn.

ON SCREEN

- Headline: **"The model forgets. The harness remembers."** Two chat apps side by side: **Without memory** (stateless, nothing saved) and **With memory** (a saved thread, thread_id finance-priya-0412). Priya sends the same messages to both.
- Turn 1: "I'm Priya from Finance. Summarise last month's travel report." Both answer: 18% over budget, two unplanned client visits and higher airfares.
- Turn 2: "And without those two visits?" Stateless: "Which visits do you mean? I can't see any earlier messages." With memory: "Without those two visits, travel would have been 3% under budget."
- Turn 3 asks for a two-line note to the team; only the memory side can write it.
- **Come back tomorrow**: the stateless app starts a new conversation and is lost; the memory app reloads the thread from our database and carries on.
- Under each chat: **what the model receives this turn** (chips for the system prompt and messages), with a token count and a small per-turn chart. Token counts are illustrative.
- Three facts: the model does not learn (memory is re-sent text, kept in our own database) · memory costs tokens (the harness trims or summarises long threads) · two kinds of memory (short-term thread, long-term facts in a separate store).

WHAT TO SAY

- "Here is Priya from Finance using two apps. Same model, same questions. The only difference is the software around it."
- After turn 2: "The left app has no idea what 'those two visits' means. The model only ever sees what it is sent this turn. The right app re-sends the whole conversation, so it knows."

- Point at the chips: “Look at what the model receives. On the left: just the new message. On the right: the system prompt plus every earlier message. Memory is text we re-send, not something the model learns.”
- After Come back tomorrow: “The next morning, the thread is restored from our own database by its thread ID, so it picks up exactly where it left off.”
- “Memory costs tokens, so long threads get trimmed or summarised, and it is all stored under our retention policy: delete the thread and the memory is gone.”

WHAT TO CLICK

1. On arrival turns 1 → 2 → 3 **play by themselves**.
2. Click **Come back tomorrow** for the next-morning comparison (yesterday’s turns fold into a “Yesterday · 4 earlier messages” pill on the memory side).
3. **Next message** steps through manually (and stops the auto-play); **Replay** starts over.

FACTS TO HAVE READY

- LangGraph memory: a checkpointer (InMemorySaver for development, SqliteSaver on one machine, PostgresSaver in production); each call names its thread_id; state is saved after every step; add_messages appends to the list.
- trim_messages or a summarisation node keeps threads inside the context window; long-term memory is a separate store (InMemoryStore, PostgresStore) with per-user namespaces.
- Prompt caching (prefix caching in vLLM and llama.cpp) makes re-sending an unchanged start of the thread cheap.

Into the next slide. “Lots of new words so far. Here are the forty you will hear in every AI meeting.”

QUESTIONS THIS SLIDE MAY RAISE

Does memory mean the model is learning about Priya?

> No. The model is unchanged. The harness stores the conversation and re-sends it. Long-term facts (like “Priya works in Finance”) sit in a separate store we control.

Where are conversations stored, and for how long?

> In our own database, under our retention policy. Deleting a thread deletes the memory.

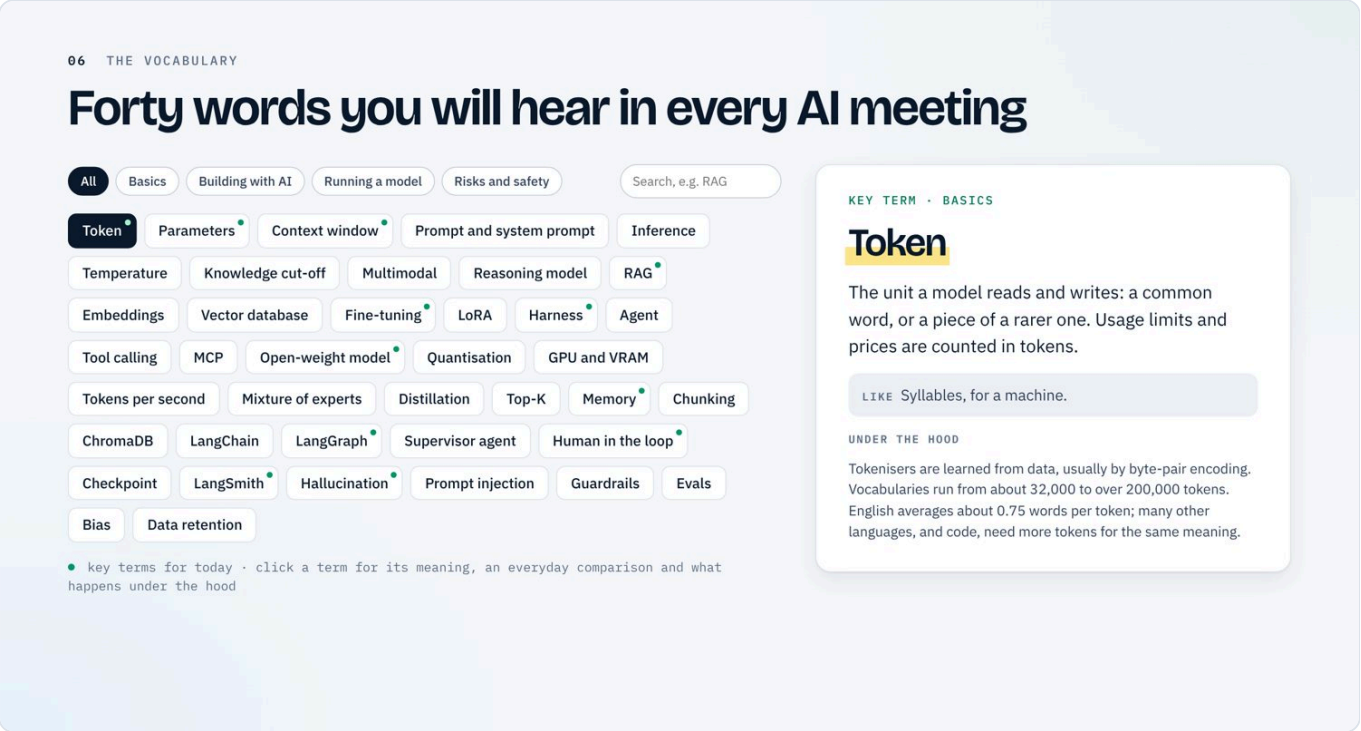
Doesn’t re-sending everything get expensive?

> Yes, it costs tokens each turn. The harness trims or summarises long threads, and prefix caching makes the unchanged start cheap.

SLIDE 8

The vocabulary

Chapter 06 · suggested time 0:20



Purpose. A reference slide: forty terms, each with a plain meaning, an everyday comparison and what happens under the hood.

ON SCREEN

- Headline: **“Forty words you will hear in every AI meeting.”** Filter chips (All · Basics · Building with AI · Running a model · Risks and safety), a search box, and the term chips. Key terms for this deck are marked with a dot.
- Clicking a term shows its meaning, an everyday comparison (“Like…”) and an “Under the hood” paragraph.

WHAT TO SAY

- “You don’t need to read these now. This is the cheat sheet: every term has a plain meaning, an everyday comparison and a technical line. Come back to it in Q&A.”
- Optionally click two: **Hallucination** and **Harness**.

WHAT TO CLICK

1. Click a filter or a term; type in the search box to filter. In the search box, Page Up/Down still change slide and Escape leaves the box.

FACTS TO HAVE READY

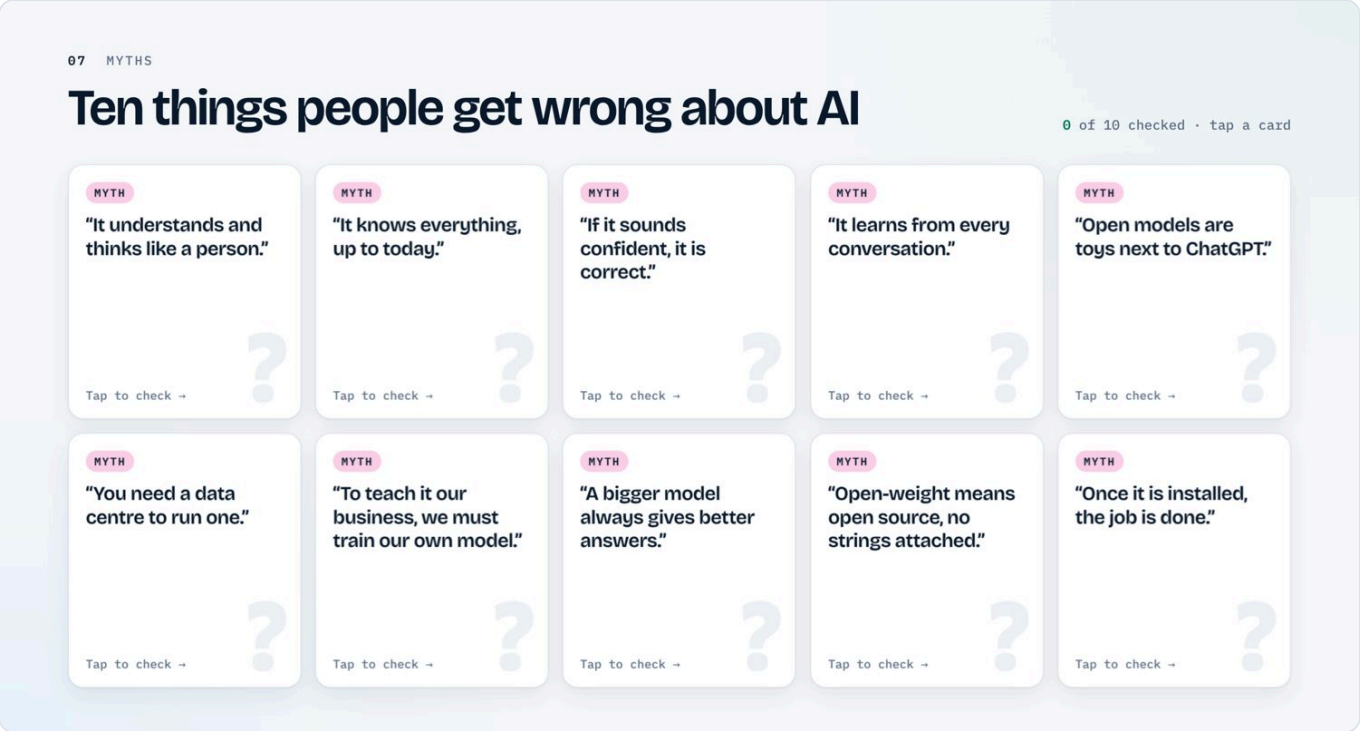
- All forty terms, with their definitions and comparisons, are in Appendix A of these notes.

Into the next slide. “And here are ten things people get wrong.”

SLIDE 9

Myths

Chapter 07 · suggested time 0:45



Purpose. Pre-empt the common objections and misunderstandings in one place.

ON SCREEN

- Headline: **"Ten things people get wrong about AI."** Ten cards, each a myth; tap a card to flip it to the reality. A counter tracks how many are checked.

WHAT TO SAY

- Pick three or four that fit your audience, for example: "If it sounds confident, it is correct" · "It learns from every conversation" · "Open models are toys next to ChatGPT" · "To teach it our business, we must train our own model".
- "Open models have closed most of the gap: OpenAI's own open-weight gpt-oss-120b reported near-parity with its o4-mini on core reasoning tests. The paid models still lead on the hardest problems."

WHAT TO CLICK

1. Click (tap) a card to flip it; the counter at the top counts up.

FACTS TO HAVE READY

- The ten myths and realities:
- 1 It understands and thinks like a person → it predicts likely text extremely well and can reason usefully, but has no intentions, awareness or memory of you beyond what the software gives it.
- 2 It knows everything, up to today → knowledge stops at a cut-off and it has never seen our private documents; RAG fills that gap.
- 3 If it sounds confident, it is correct → fluency is not accuracy; answers from our documents with sources are far easier to check.
- 4 It learns from every conversation → frozen after training; apps may remember through a separate memory feature; a model we host learns nothing unless we retrain it.

- 5 Open models are toys → gpt-oss-120b (August 2025) reported near-parity with o4-mini on core reasoning tests; paid models still lead on the hardest problems.
- 6 You need a data centre → one well-equipped server can run a strong open-weight model for a department; smaller models need a single GPU.
- 7 To teach it our business, we must train our own model → almost never; RAG is cheaper, always up to date and shows sources; fine-tuning changes style; training from scratch costs tens of crores of rupees or more.
- 8 A bigger model always gives better answers → for our business, the right documents matter more; a mid-size model with good search often beats a giant one without it.
- 9 Open-weight means open source, no strings attached → weights are downloadable, training data usually is not, and licences differ (Apache 2.0 vs Llama's conditions); Legal should review ours.
- 10 Once installed, the job is done → it needs an owner: testing, moving to better models, monitoring and security updates.

Into the next slide. "Enough about the machine. Where does AI actually help us?"

QUESTIONS THIS SLIDE MAY RAISE

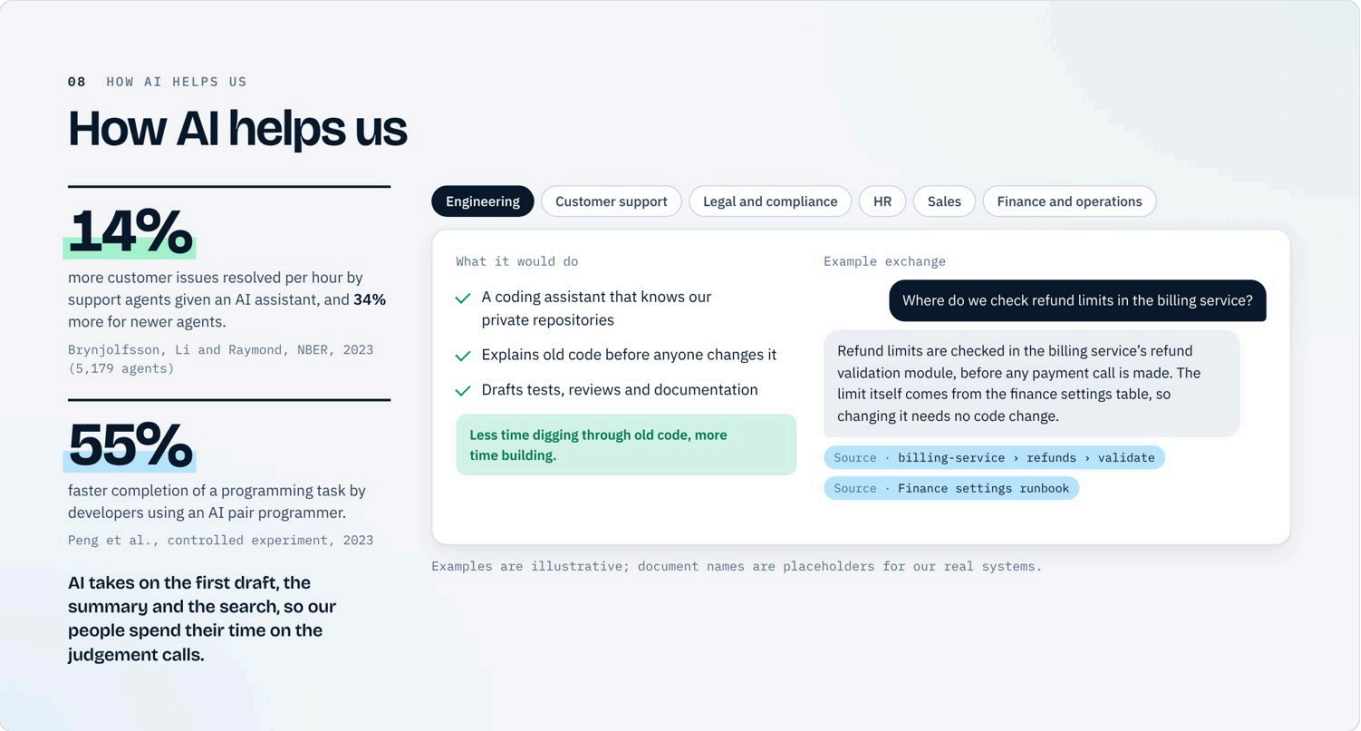
Which licence does gpt-oss use?

> Apache 2.0, a permissive licence. Legal should still review whichever model we choose.

SLIDE 10

How AI helps us

Chapter 08 (part 2: The benefits of AI) · suggested time 0:50



Purpose. Ground the benefits in published evidence and in concrete examples for each department.

ON SCREEN

- Headline: **“How AI helps us.”** Two evidence figures: **14%** more customer issues resolved per hour by support agents with an AI assistant, and **34%** more for newer agents (Brynjolfsson, Li and Raymond, NBER, 2023, 5,179 agents); **55%** faster completion of a programming task with an AI pair programmer (Peng et al., controlled experiment, 2023).
- Claim: “AI takes on the first draft, the summary and the search, so our people spend their time on the judgement calls.”
- Department tabs: Engineering, Customer support, Legal and compliance, HR, Sales, Finance and operations. Each shows what AI would do, a one-line benefit and an example exchange that types out with its sources.

WHAT TO SAY

- “Two pieces of evidence. In a study of over five thousand support agents, an AI assistant raised issues resolved per hour by 14%, and by 34% for the newest agents: it spreads the know-how of the best people. And developers with an AI pair programmer finished a task 55% faster.”
- Click two departments relevant to the room: “For Legal: first-pass review of contracts against our standard terms. For HR: policy answers that quote the exact clause.”
- “The pattern is the same everywhere: AI does the first draft, the summary and the search; people make the judgement calls.”

WHAT TO CLICK

1. Click a department tab; the example answer types out and its sources appear. The card stays the same size for every department.

FACTS TO HAVE READY

- Engineering: less time digging through old code · Customer support: faster, consistent answers · Legal: hours of reading become minutes of checking · HR: answers at once, without waiting for HR · Sales: more time with clients, less on first drafts · Finance and operations: variances explained in minutes, not days.
- Examples and document names are illustrative placeholders for our real systems.

Into the next slide. “To get those benefits safely, the model needs software around it. That is the harness.”

QUESTIONS THIS SLIDE MAY RAISE

Are those studies relevant to us?

> They are two of the most-cited controlled studies of AI at work: one in customer support, one in software development. Our own pilot would measure our own numbers.

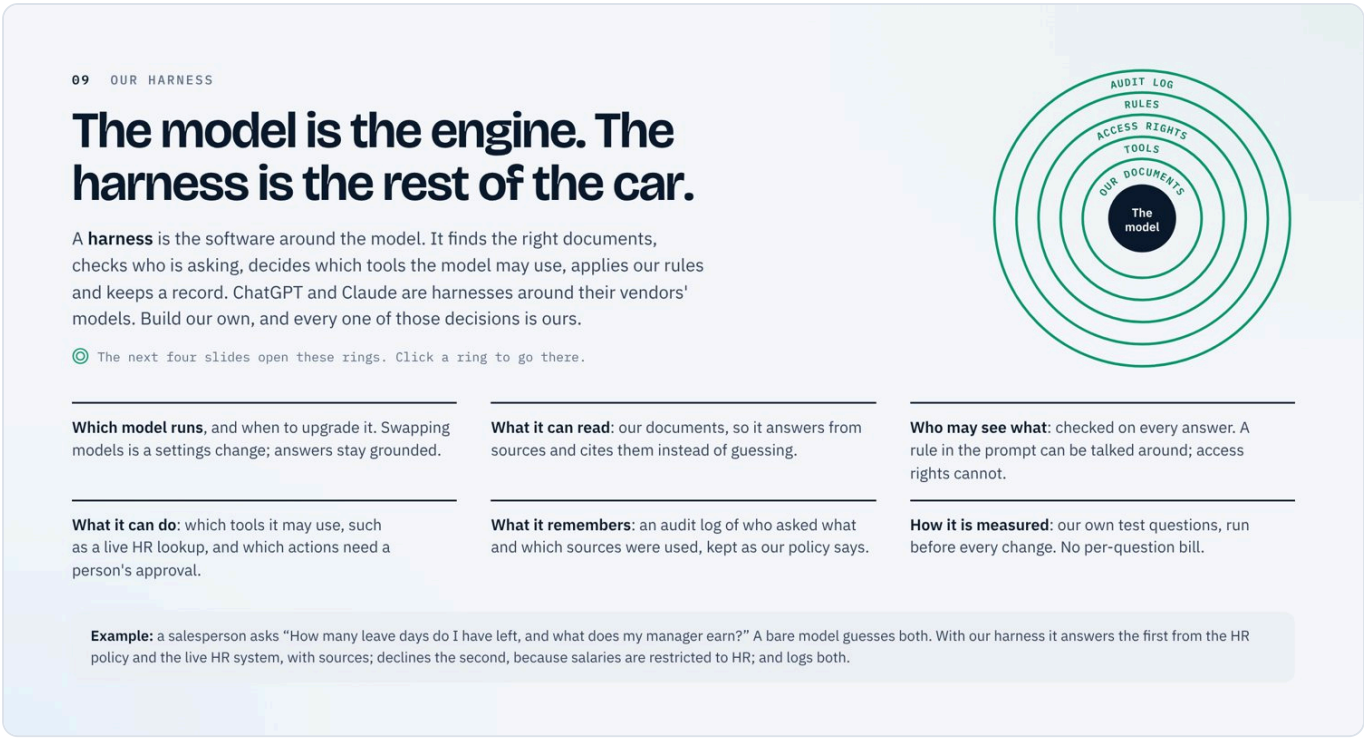
Will it replace jobs?

> The evidence here is about doing the same work faster, especially for newer staff. The examples keep a person responsible for the judgement and the decision.

SLIDE 11

Our harness

Chapter 09 (part 3: What we have built) · suggested time 0:50



Purpose. Introduce the harness, the software around the model, as the thing we build and control. The next four slides each open one of its rings.

ON SCREEN

- Headline: **"The model is the engine. The harness is the rest of the car."** Concentric rings around "The model": Our documents, Tools, Access rights, Rules, Audit log.
- Six controls: which model runs · what it can read · who may see what · what it can do · what it remembers · how it is measured.
- Example callout: a salesperson asks "How many leave days do I have left, and what does my manager earn?" With our harness it answers the first from the HR policy and the live HR system, with sources; declines the second, because salaries are restricted to HR; and logs both.

WHAT TO SAY

- "ChatGPT and Claude are harnesses around their vendors' models. The harness finds the right documents, checks who is asking, decides which tools the model may use, applies our rules and keeps a record."
- Read the example: "A bare model guesses both. Ours answers the first with sources, declines the second, and logs both."
- "The next four slides open these rings one by one: our documents, tools, rules and the audit log."

WHAT TO CLICK

1. Hover a ring to see where it leads; **click a ring** to jump to its slide (Our documents and Access rights → RAG; Tools → multi-agent; Rules → human in the loop; Audit log → observability).

FACTS TO HAVE READY

- Target architecture (Engineer's note): an inference server (vLLM for many users) behind an OpenAI-compatible API; ingestion with connectors, chunking, embeddings, hybrid search and a reranker; a policy layer enforcing single sign-on identity and document permissions at retrieval time; tools through MCP servers with an allow-list and human approval for write actions; prompt-injection defences; an evaluation set run on every change; tracing of every request. Swapping the model is a configuration change.

Into the next slide. "First ring: our documents."

QUESTIONS THIS SLIDE MAY RAISE

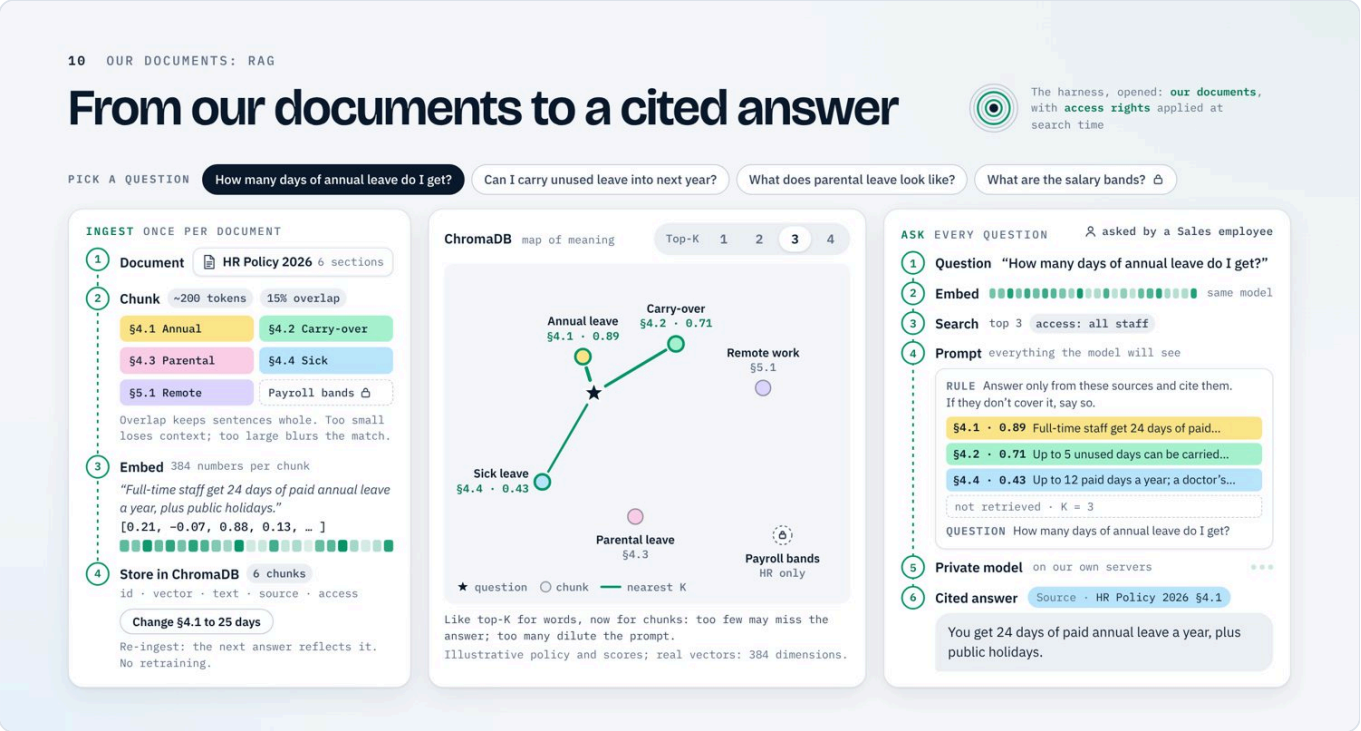
Is "a rule in the prompt" not enough to protect salaries?

> No. A prompt rule can be talked around. Access rights are enforced by the search itself, using the person's sign-in groups, so restricted text never reaches the model.

SLIDE 12

Our documents: RAG

Chapter 10 · suggested time 1:30



Purpose. Show RAG end to end: ingesting a document into ChromaDB and answering a question from it with a citation, including access control and updating a document without retraining.

ON SCREEN

- Headline: **"From our documents to a cited answer."** A harness badge: our documents, with access rights applied at search time.
- **Ingest** (once per document): HR Policy 2026 (6 sections) → **Chunk** (~200 tokens, 15% overlap) → **Embed** (384 numbers per chunk) → **Store in ChromaDB** (id · vector · text · source · access).
- **ChromaDB map of meaning** (centre): the chunks as dots; the question lands as a star and lines reach its nearest K chunks with similarity scores. Top-K buttons 1–4.
- **Ask** (every question), asked by a Sales employee: Question → Embed (same model) → Search (top K, access: all staff) → **Prompt** (a rule: answer only from these sources and cite them, plus the K chunks and the question) → Private model (on our own servers) → **Cited answer** ("You get 24 days of paid annual leave a year, plus public holidays.", source HR Policy 2026 §4.1).
- Question chips: annual leave · carrying leave over · parental leave · **salary bands** (locked: HR only).

WHAT TO SAY

- "This is the fix from slide 4 in action. First, once per document: we cut it into chunks, turn each chunk into 384 numbers, the same kind of embedding we saw on slide 5, and store them in ChromaDB."
- "Then for every question: the question is embedded with the same model, we find the nearest chunks on the map of meaning, and we hand only those to the model with a rule: answer only from these, and cite them."
- Click **What are the salary bands?**: "The payroll chunk is actually the closest match, but this person is in Sales. The search filters it out before the model ever sees it, so the answer is a polite 'no permitted source'."
- Click **Change §4.1 to 25 days**: "Policies change. We re-ingest just that chunk, and the very next answer says 25. No retraining."

- Top-K: “The same top-K idea as slide 6, now for passages: too few may miss the answer, too many dilute the prompt.”

WHAT TO CLICK

1. On arrival the document is ingested (about 3 s), then the first question plays (about 12 s in total).
2. Click a question chip to ask it; click Top-K 1–4 to change how many chunks are retrieved (the prompt always shows four slots; unused ones read “not retrieved”).
3. Click **Change §4.1 to 25 days** (and **Undo**) to show re-ingestion.
4. Click a chunk tile in the Ingest lane to see it as numbers.

FACTS TO HAVE READY

- Chunking: usually 300–800 tokens with 10–20% overlap; with all-MiniLM-L6-v2 (reads only the first 256 word pieces) we keep chunks near 200 tokens.
- Embeddings: all-MiniLM-L6-v2 gives 384 numbers and runs on a CPU; stronger options include EmbeddingGemma and nomic-embed-text (768 numbers). Documents and questions must use the same model.
- ChromaDB: open source (Apache 2.0), runs in the app or as its own server on our machines; access control is a metadata filter inside the search, with groups from the person’s sign-in, never from the model; changed documents are upserted by id.
- Getting it right: hybrid search (BM25 + vectors), a reranker, and a test set of real questions (hit rate at K).
- The policy, the scores and the map are illustrative.

Into the next slide. “Search is one tool. What happens when a request needs several tools, or several steps?”

QUESTIONS THIS SLIDE MAY RAISE

What if the answer isn’t in our documents?

> The prompt’s rule tells the model to say so. The salary example shows the same behaviour when the right document is off-limits.

How do we keep it up to date?

> Re-ingest changed documents (upsert their chunks by id, delete removed ones). Nothing is retrained.

Can it read PDFs, scans, spreadsheets?

> Text documents ingest directly; scanned PDFs need OCR first, and tables need care when chunking. This is part of building the connectors.

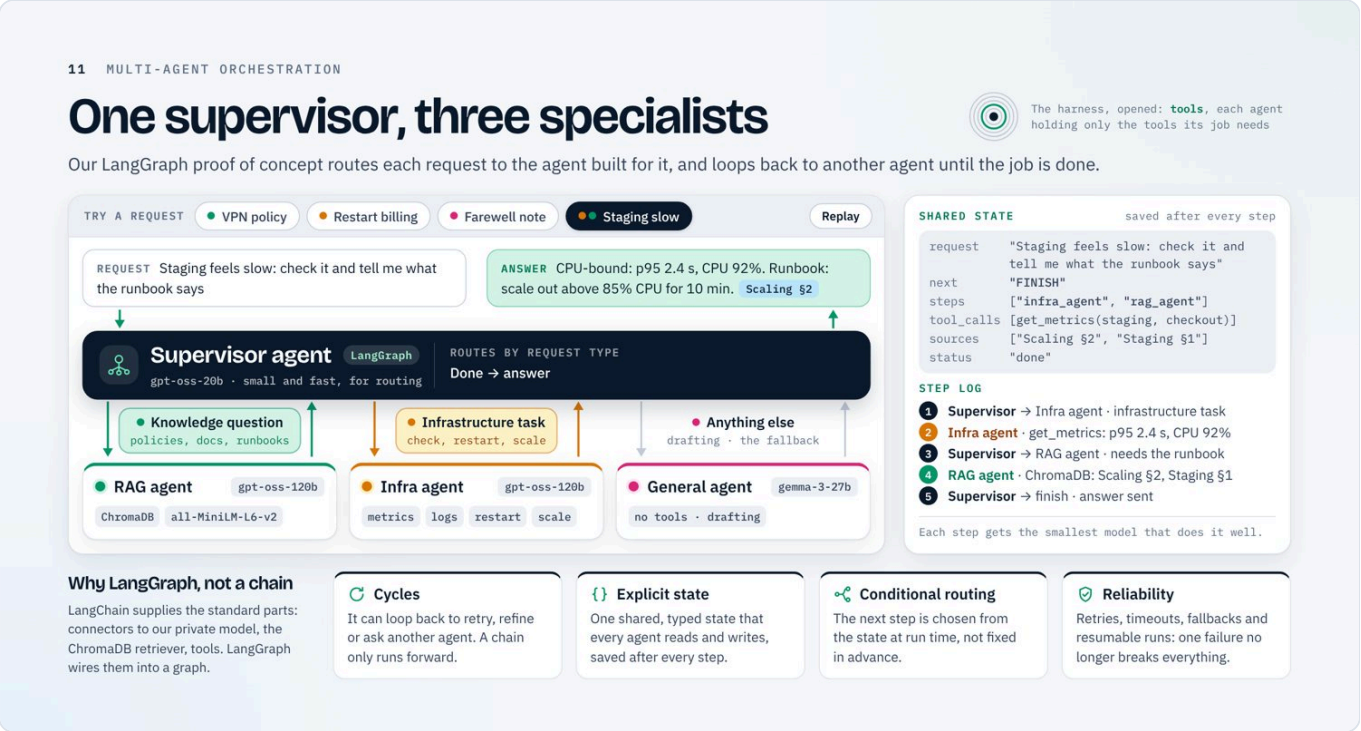
Why ChromaDB? Will it scale?

> It is simple, open source and runs on our own machines, which suits a proof of concept. For larger scale there are PostgreSQL with pgvector, Qdrant, Milvus and OpenSearch; LangChain makes the store swappable.

SLIDE 13

Multi-agent orchestration

Chapter 11 · suggested time 1:30



Purpose. Show the LangGraph proof of concept: a supervisor agent that classifies each request by type and routes it to the RAG, Infra or General agent, looping until the job is done.

ON SCREEN

- Headline: **“One supervisor, three specialists.”** Harness badge: tools, each agent holding only the tools its job needs.
- The navy **Supervisor agent** (LangGraph, gpt-oss-20b, small and fast for routing) with a live readout of its routing decision. Three routing rules: **Knowledge question** (policies, docs, runbooks) → RAG agent · **Infrastructure task** (check, restart, scale) → Infra agent · **Anything else** (drafting, the fallback) → General agent.
- Agents: **RAG agent** (gpt-oss-120b; ChromaDB, all-MiniLM-L6-v2) · **Infra agent** (gpt-oss-120b; metrics, logs, restart, scale) · **General agent** (gemma-3-27b; no tools, drafting).
- Right: the **shared state** (request, next, steps, tool_calls, sources, status), saved after every step, and a numbered step log. Footer: each step gets the smallest model that does it well.
- Bottom: why LangGraph, not a chain: cycles · explicit state · conditional routing · reliability.

WHAT TO SAY

- “Our proof of concept is a team: a supervisor and three specialists. Every request goes to the supervisor first. It decides what type of request it is and routes it.”
- During the auto-run: “‘Staging feels slow, check it and tell me what the runbook says.’ The supervisor says: infrastructure task, so the Infra agent reads the metrics, CPU at 92%. Back to the supervisor: it still needs the runbook, a knowledge question, so the RAG agent searches ChromaDB. Then: done, and a combined answer with its source.”
- “That loop back is why we use a graph rather than a straight chain: it can revisit, keep one shared state, choose the next step at run time and recover from failures.”

- Click **Restart billing**: “Changes need permission. Even on staging, the restart pauses until an admin approves.” Then click **Approve as admin**.
- On models: “Routing is easy, so it runs on the small gpt-oss-20b; tool use and grounded answers on gpt-oss-120b; drafting on Gemma 3 27B.”

WHAT TO CLICK

1. On arrival it plays the two-hop **Staging slow** run by itself (about 10 s).
2. Request chips: **VPN policy** (RAG only, with a citation) · **Restart billing** (Infra; **pauses for “Approve as admin”**: you must click the green button in the answer box to continue) · **Farewell note** (General agent, no tools) · **Staging slow** (Infra then RAG). **Replay** reruns the current request.
3. Leaving mid-run stops it; it restarts from the beginning when you come back.

FACTS TO HAVE READY

- LangGraph StateGraph with a typed state (messages with the add_messages reducer, and next); the supervisor uses structured output so it can only answer rag_agent, infra_agent, general_agent or FINISH; conditional edges route on next; every agent has an edge back to the supervisor (the loop).
- Sub-agents are prebuilt ReAct agents (create_react_agent; create_agent in LangChain 1.0). Infra tools wrap approved runbooks and APIs: get_metrics, get_logs, restart_service, scale_service.
- A checkpointer gives memory and resumable runs; recursion_limit stops runaway loops; interrupt before infra write actions hands the decision to a person; LangSmith traces every node.
- LangChain’s ChatOpenAI(base_url=…) points every node at our own model server’s OpenAI-compatible API.
- Figures (p95 2.4 s, CPU 92%) and runbook passages are illustrative.

Into the next slide. “That approval step is the next ring: rules. Let’s look at it properly, on production.”

QUESTIONS THIS SLIDE MAY RAISE

What stops an agent from doing something dangerous?

> Each agent only has the tools its job needs (an allow-list), read-only actions run on their own, and any change waits for a person: an admin on staging, the owner on production (next slide). Everything is traced.

What if the supervisor routes wrongly?

> It uses structured output, so it can only choose one of the four answers, and the trace shows every routing decision for review. A wrong route usually comes back to the supervisor, which can try another agent.

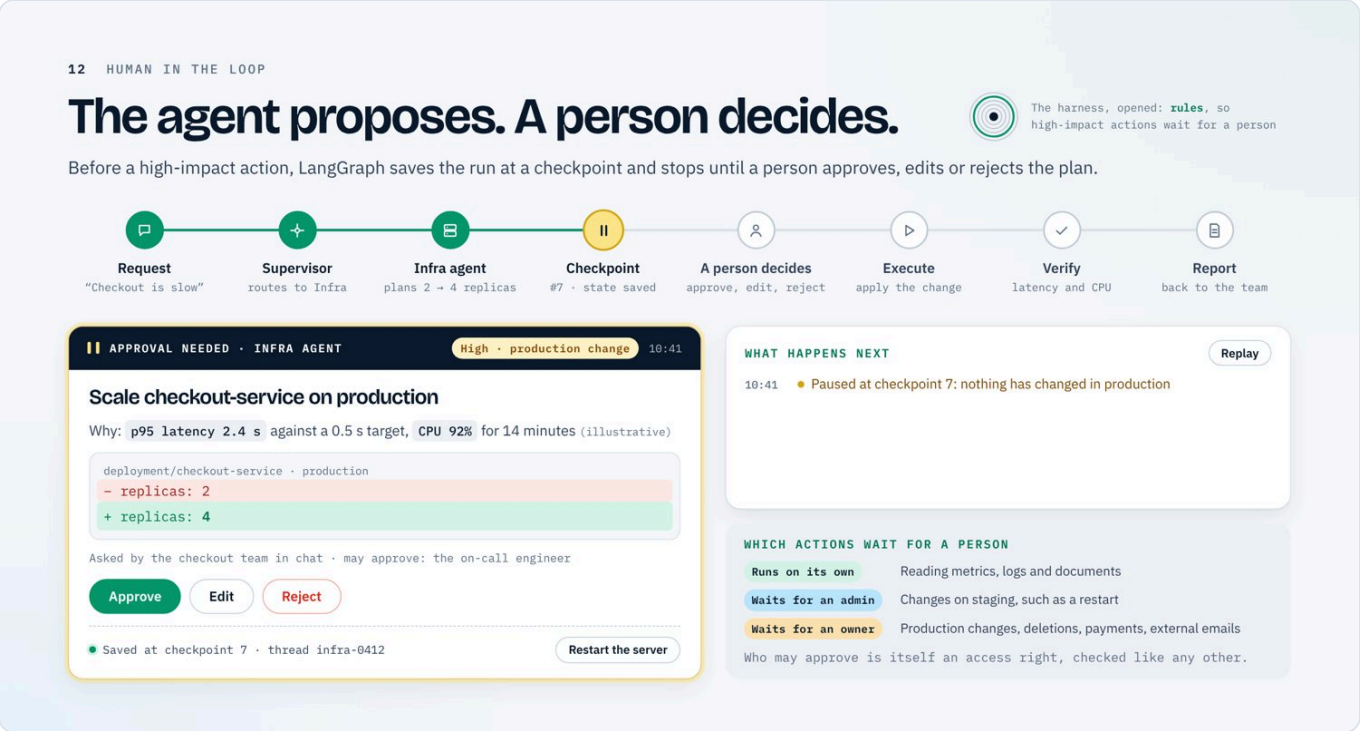
Why three different models?

> Cost and speed: each step gets the smallest model that does it well. Routing needs a quick, small model; tool use and grounded answers benefit from the larger one.

SLIDE 14

Human in the loop

Chapter 12 · suggested time 1:00



Purpose. Show the approval checkpoint: the agent proposes a production change, the run is saved and waits, a person approves, edits or rejects, and the run resumes safely, even after a server restart.

ON SCREEN

- Headline: **“The agent proposes. A person decides.”** Harness badge: rules, so high-impact actions wait for a person.
- A timeline: Request (“Checkout is slow”) → Supervisor → Infra agent (plans 2 → 4 replicas) → **Checkpoint** (#7, state saved) → A person decides → Execute → Verify → Report.
- An approval card: “Scale checkout-service on production”, why (p95 2.4 s against a 0.5 s target, CPU 92% for 14 minutes, illustrative), the diff (replicas 2 → 4), who may approve (the on-call engineer), and **Approve · Edit · Reject**. Footer: saved at checkpoint 7, thread infra-0412, and **Restart the server**.
- “What happens next” log, and a policy panel: **Runs on its own** (reading metrics, logs and documents) · **Waits for an admin** (changes on staging, such as a restart) · **Waits for an owner** (production changes, deletions, payments, external emails). “Who may approve is itself an access right.”

WHAT TO SAY

- “Before a high-impact action, the run is saved at a checkpoint and stops. Nothing has changed in production. It waits for a named person.”
- Click **Restart the server**: “Even if the server restarts while it waits, nothing is lost: the checkpoint lives in a database.”
- Click **Edit**, change to 3 replicas, then approve: “The person can edit the plan, not just say yes or no. The decision is stamped with who made it.”
- Or click **Reject**: “Rejected: no change, and the agent will alert again if the problem persists.”
- Point at the policy: “Reading runs on its own; staging changes wait for an admin; production changes, deletions, payments and outside emails wait for their owner.”

WHAT TO CLICK

1. On arrival the run plays up to the checkpoint and **waits**.
2. **Approve**, **Edit** (a stepper, 3–6 replicas, starting at the proposed 4; “Approve n replicas”), **Reject**, **Restart the server** (shows the run restored from checkpoint 7), **Replay** (start again).
3. The stamp names the person, for example “Edited to 5 replicas by the on-call engineer · 10:42 · logged”.

FACTS TO HAVE READY

- Dynamic interrupt() inside the node: the caller gets an __interrupt__ payload (what the approval screen shows); resume with graph.invoke(Command(resume=...), config); the node reruns from its top, so interrupt() comes before any side effect.
- A static breakpoint (interrupt_before=["execute"]) stops before the node; an edit goes in with graph.update_state; graph.invoke(None, config) continues.
- The checkpoint lives in SQLite or Postgres, so a restart loses nothing. In production add timeouts and escalation for unanswered approvals, and record who approved.

Into the next slide. “Every one of those steps, the pause, the decision and the resume, is on the record. That is the last ring.”

QUESTIONS THIS SLIDE MAY RAISE

What if nobody approves?

> In production we would add a timeout and an escalation path. Until someone decides, nothing changes.

Who decides who can approve?

> Approval rights are an access right like any other, checked from the person’s sign-in.

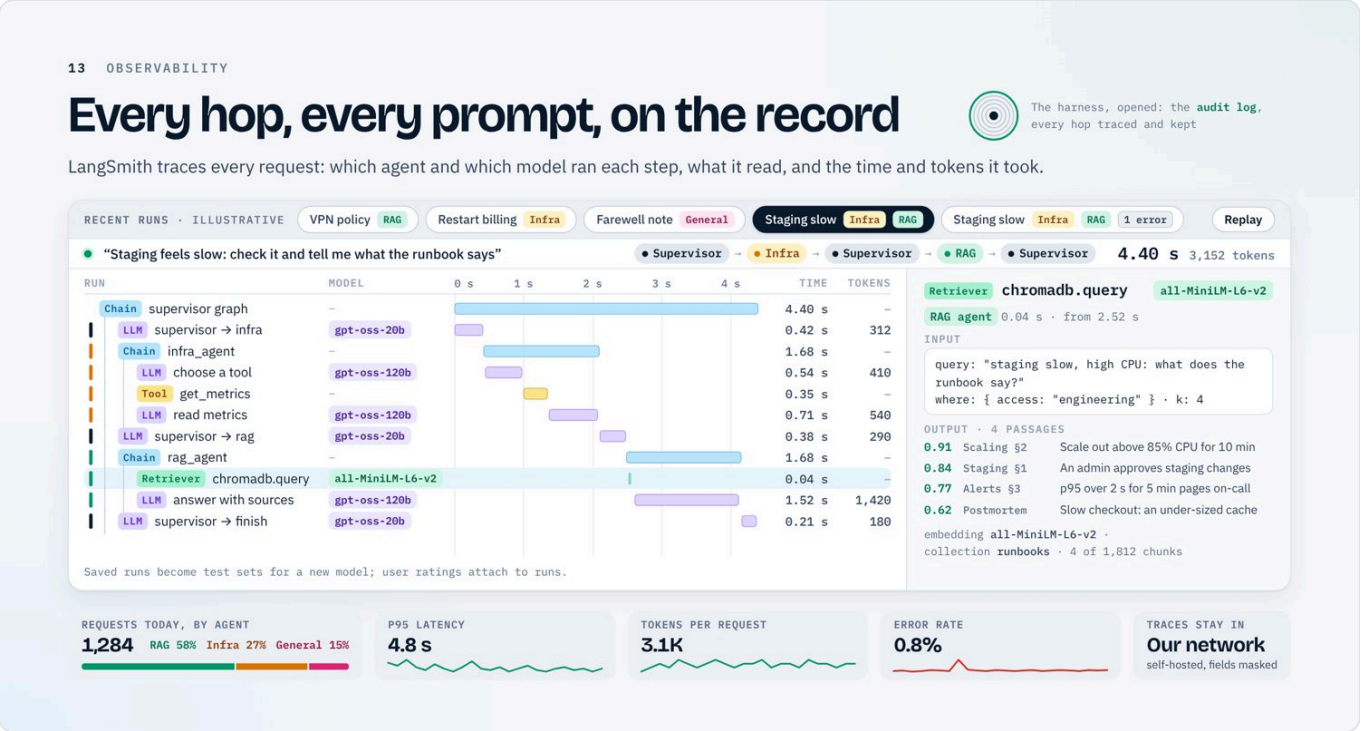
Can it do anything without asking?

> Read-only actions (metrics, logs, documents) run on their own. Every change waits for a person.

SLIDE 15

Observability

Chapter 13 · suggested time 1:00



Purpose. Show LangSmith traces: which agent and which model ran each step, what it read, how long each step took and how many tokens it used, plus a failure being debugged.

ON SCREEN

- Headline: **“Every hop, every prompt, on the record.”** Harness badge: the audit log, every hop traced and kept.
- Recent runs (illustrative): VPN policy (RAG) · Restart billing (Infra) · Farewell note (General) · Staging slow (Infra + RAG) · Staging slow with **1 error**.
- A route line (Supervisor → Infra → Supervisor → RAG → Supervisor), total time (4.40 s) and tokens (3,152).
- A trace tree with a waterfall: each step with its type (Chain, LLM, Tool, Retriever, Approval), its agent colour, **the model used** (gpt-oss-20b, gpt-oss-120b, all-MiniLM-L6-v2), time and tokens. Clicking a step opens an inspector with its input, output and metadata.
- Monitoring strip: requests today by agent (1,284: RAG 58%, Infra 27%, General 15%), p95 latency 4.8 s, tokens per request 3.1K, error rate 0.8%, and “traces stay in our network” (self-hosted, fields masked).

WHAT TO SAY

- “LangSmith records every request as a tree of steps: which agent ran, which model it used, what it read, what it sent the model, how long each step took and how many tokens it used.”
- Point at the default inspector (chromadb.query): “Here is exactly what the RAG agent retrieved for the staging question, with the similarity scores.”
- Click the **Staging slow · 1 error** run: “When something is slow, we can point at the step. Here the metrics API timed out after 10 seconds, the retry worked, and the slow step was the API, not the model.”
- Click **Restart billing**: “The admin approval from two slides ago is in the trace too.”
- “Saved runs become test sets: before we switch to a new model, we replay real questions and compare. And it is self-hosted, so traces, which contain prompts and documents, stay in our network.”

WHAT TO CLICK

1. On arrival the healthy Staging slow trace **streams in**, then the inspector opens on chromadb.query.
2. Click any run chip to load it; click any step to inspect it (a step you click while it is streaming stays selected); **Replay** streams the current run again.

FACTS TO HAVE READY

- Enable with LANGSMITH_TRACING=true and LANGSMITH_API_KEY, plus LANGSMITH_ENDPOINT for a self-hosted instance. LangChain and LangGraph runs are traced automatically; other code with the @traceable decorator.
- Dashboards and alerts (latency, tokens, errors), user feedback on runs, and datasets for evaluations.
- Self-hosted LangSmith is offered on the Enterprise plan; inputs and outputs can be masked; traces can be exported over OpenTelemetry. Open-source alternative: Langfuse (self-hostable).
- The trace and every figure on this slide are illustrative.

Into the next slide. “Let me pull it all together.”

QUESTIONS THIS SLIDE MAY RAISE

Is LangSmith free?

> There is a hosted service with free and paid tiers, but self-hosting (which keeps traces in our network) is on the Enterprise plan. Langfuse is an open-source alternative we can self-host.

Don’t traces contain sensitive data?

> Yes, prompts and documents. That is why we self-host and mask sensitive fields.

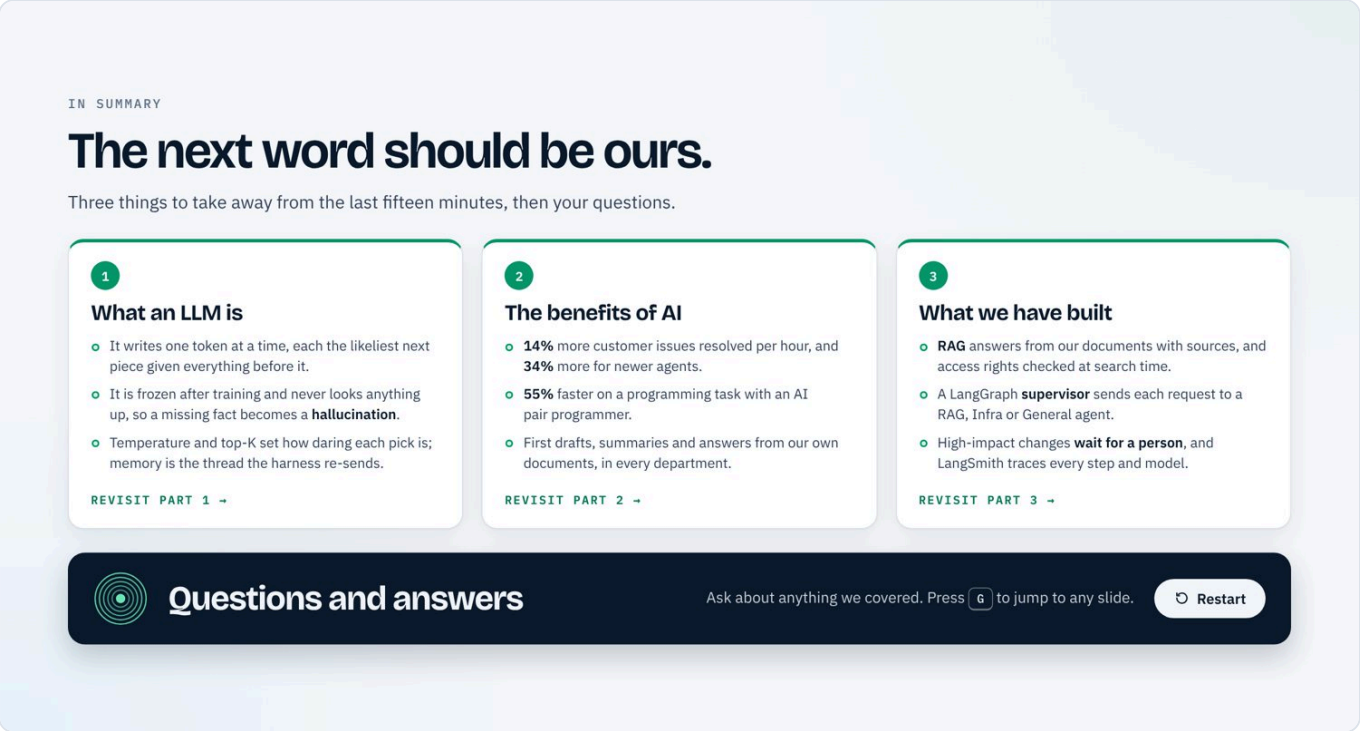
Are these numbers real?

> No: the trace and the monitoring figures are illustrative, and the slide says so. A pilot would produce real ones.

SLIDE 16

Summary and questions

Chapter (closing slide) · suggested time 0:30 + Q&A



> **Purpose.** Bookend the talk with the opening line, recap the three parts, and open the floor.

ON SCREEN

- Headline: **“The next word should be ours.”** (the title slide’s line again).
- Three cards: **What an LLM is** (one token at a time; frozen, so missing facts become hallucinations; temperature and top-K; memory is the re-sent thread) · **The benefits of AI** (14% / 34%, 55%, first drafts, summaries and answers in every department) · **What we have built** (RAG with sources and access rights; a LangGraph supervisor routing to RAG, Infra and General agents; changes wait for a person; LangSmith traces every step and model).
- Each card has a **Revisit part N →** link.
- A navy **Questions and answers** banner with the harness rings lighting up slowly, “Press G to jump to any slide”, and a **Restart** button.

WHAT TO SAY

- “Three things to take away. An LLM writes one token at a time and is frozen after training, so it needs our documents to be right. AI measurably speeds up real work. And we have built the system around the model: our documents with access rights, agents that route work, people approving every change, and a full record of every step.”
- “The next word should be ours. Questions?”

WHAT TO CLICK

1. During Q&A: press **G** for the slide list, or click **Revisit part 1/2/3** to jump back.
2. **Restart** goes back to slide 1 (so does the Home key).

QUESTIONS AND ANSWERS

Questions people may ask

Grouped by theme. Slide-specific questions are also on each slide's page.

Purpose and next steps

What are you asking us for?

> Today is a show and tell. If the room wants a next step, the natural one is a small pilot: one department, its documents, twenty to thirty users, and an honest measurement of time saved and accuracy (outline in Appendix B).

How long would a pilot take?

> The earlier version of this deck outlined about 12 weeks: 2 to choose a department and agree success measures, 6 to build (one GPU server, an open-weight model with RAG inside our harness, access rights and an audit log), and 4 to measure.

Who would own and run it?

> It needs an owner in IT: testing, moving to better models as they appear, monitoring and security updates. The earlier cost notes assumed roughly a quarter to half of one engineer's time, plus support contracts.

What is real today and what is a mock-up?

> **Confirm this yourself before presenting.** The slides describe a LangGraph proof of concept; the traces, metrics, policies and example documents on the slides are labelled illustrative. Be ready to say exactly which parts run today.

Why our own system, not a subscription

Why not just use ChatGPT Enterprise or Copilot?

> They are excellent for general work, and the best of them still lead on the hardest problems. Running our own matters where our data and systems are involved: documents and prompts stay in our network, access rights come from our sign-in, every step is in our audit log, tools connect directly to our systems, and there is no per-seat bill as use grows. The two can coexist.

What does it cost?

> The cost slide was removed from this version. If pressed, Appendix B has the earlier deck's illustrative calculator defaults (for example, a server bought once for about ₹30 lakh plus about ₹1.25 lakh a month to run, against subscriptions of ₹2,900 per person per month for 150 people). Quote them only as illustrations and offer to come back with real quotes.

Are open models really good enough?

> For everyday work (drafting, summarising, answering from our documents, help with code), yes. OpenAI's open-weight gpt-oss-120b reported near-parity with its o4-mini on core reasoning tests. The strongest paid models still lead on the hardest tasks.

Why did you choose these particular models?

> gpt-oss-120b and gpt-oss-20b are OpenAI's open-weight models under the permissive Apache 2.0 licence; gpt-oss-120b is a mixture of experts (about 5.1B of 117B parameters active per token) and fits on one 80 GB GPU. Gemma 3 27B (Google) drafts well. The examples deliberately stick to open-weight models from US and European providers; whichever we choose, Legal reviews the licence.

Can we switch models later?

> Yes. The harness talks to models through one OpenAI-compatible API, so swapping a model is a configuration change. Saved traces let us test a new model on real questions before switching.

Data, privacy and security

Does any of our data leave the network?

> Not with this design: the models run on our own servers, ChromaDB runs on our machines, and LangSmith can be self-hosted with sensitive fields masked.

Can someone get an answer from a document they are not allowed to see?

> No. Access rights are applied inside the search, using the groups from the person's sign-in, never from the model. Restricted chunks never reach the prompt (the salary-bands example on slide 12).

What about prompt injection, for example a malicious document telling the AI to do something?

> Retrieved text is treated as data, not instructions; agents only have allow-listed tools; and every change waits for a person. These defences are part of the target architecture.

How long are conversations kept?

> Threads are stored in our own database under our retention policy. Deleting a thread deletes the memory.

Could the AI change production by itself?

> No. Read-only actions run on their own; staging changes wait for an admin; production changes, deletions, payments and external emails wait for their owner. Approval rights are themselves an access right, and every decision is recorded.

Accuracy and quality

How do we know the answers are right?

> Three ways: answers come from our documents and cite them, so people can check; we keep a test set of real questions with known answers and run it on every change; and every answer is traced, so a wrong one can be traced to the step that caused it.

Will it still hallucinate?

> Less, not never. RAG, a low temperature and the instruction to say when the sources do not cover a question reduce it sharply, and citations make the rest easy to spot.

What temperature would we use?

> About 0 to 0.3 for factual answers, code and RAG; around 0.7 for chat; higher only for brainstorming.

Does it understand Hindi and other languages?

> Modern models read and write many languages, including Hindi. Other languages use more tokens for the same meaning, so they cost a little more and fill the context faster. We would test on our own content.

Technology

What hardware do we need?

> gpt-oss-120b fits on a single 80 GB data-centre GPU; gpt-oss-20b runs within about 16 GB. More users need more GPUs or a batching inference server such as vLLM. Embeddings (all-MiniLM-L6-v2) run on an ordinary CPU.

How fast is it?

> People read about 5 tokens a second; a good private setup writes 30 to 100+. A 500-word answer is about 650 tokens.

Why LangGraph rather than plain LangChain?

> LangChain supplies the parts (model connectors, the ChromaDB retriever, tools). LangGraph wires them into a graph that can loop, keeps one shared state saved after every step, chooses the next step at run time, and can pause for approval and resume after a restart.

Do we need to fine-tune or train our own model?

> Almost certainly not to start. RAG gives it our knowledge without retraining. Fine-tuning (for example LoRA, hours on one GPU) helps with style, format or narrow tasks if tests show a need. Training from scratch is not needed.

What is MCP?

> The Model Context Protocol: a standard way to connect a model to tools and data sources, so tools are written once and reused. Our harness would expose tools through MCP servers with an allow-list.

What happens when a document changes?

> We re-ingest just that document’s chunks (upsert by id) and the next answer reflects it; nothing is retrained (the “Change §4.1 to 25 days” button on slide 12).

Does the model learn from our conversations?

> No. It is frozen. Memory is the conversation re-sent by the harness, stored in our database.

APPENDIX A

The forty terms (slide 8)

TERM	MEANING	LIKE
Token	The unit a model reads and writes: a common word, or a piece of a rarer one. Usage limits and prices are counted in tokens.	Syllables, for a machine.
Parameters	The billions of numbers inside a model that were adjusted during training. Together they hold everything it learned.	The strength of the connections in a brain.
Context window	How much text the model can consider at once: the instructions, the conversation so far and any documents supplied, measured in tokens.	A desk. Only what fits on the desk gets used.
Prompt and system prompt	The prompt is what a person sends. The system prompt is standing instructions set by whoever runs the tool: role, rules, tone and format.	The briefing a new employee gets on day one, then the day's requests.
Inference	Using a trained model to produce an answer. It happens every time anyone asks a question.	Training is school. Inference is doing the job.
Temperature	A setting that controls how adventurous the model is when it picks each next word. Low is predictable, high is creative.	A careful accountant versus a brainstorming session.
Knowledge cut-off	The date the training data ends. The model knows nothing after it unless the harness gives it documents or search results.	A well-read colleague who has been off-grid since that date.
Multimodal	A model that handles more than text: images, scanned documents, charts, audio, sometimes video.	A colleague who reads the attachment, not only the email.
Reasoning model	A model trained to work through a problem step by step before answering. Slower, but much better at maths, code and planning.	Showing your working in an exam.
RAG	Retrieval-augmented generation. Before answering, the system searches our documents for the relevant passages and gives them to the model with the question, so it answers from them and can cite them.	An open-book exam instead of answering from memory.
Embeddings	A way to turn a piece of text into a list of numbers that captures its meaning, so software can find passages that mean the same thing even when the words differ.	Map coordinates for meaning: similar ideas sit close together.
Vector database	A database that stores embeddings and quickly finds the ones closest to a question. It is the search engine behind RAG.	A library catalogue organised by topic instead of title.
Fine-tuning	Further training an existing model on our own examples, to change its behaviour, tone or output format.	On-the-job training for a graduate who already has a degree.
LoRA	A cheap way to fine-tune. Instead of changing every parameter, a small add-on is trained: hours on one GPU instead of weeks on many.	Clip-on lenses instead of new glasses.
Harness	All the software around a model that turns it into a useful tool: instructions, memory, access to documents and tools, permissions and logging.	The model is the engine; the harness is the rest of the car.

TERM	MEANING	LIKE
Agent	A model inside a harness that works through a task in steps: search, read, calculate, write, check its own work, and repeat until done.	An assistant you give a goal, not a single instruction.
Tool calling	The model's ability to ask the harness to run something, such as a search, a database query or a calculation, and then use the result.	Asking a colleague to check the system while you draft the reply.
MCP	Model Context Protocol: an open standard for connecting AI tools to data sources and apps, so a connector is built once and works with many models.	USB for AI: one plug shape for many devices.
Open-weight model	A model whose trained parameters are published for anyone to download and run on their own hardware.	Getting the recipe, not just the meal.
Quantisation	Storing each parameter with fewer bits, for example 4 instead of 16, so the model needs about a quarter of the memory for a small loss in quality.	A high-quality MP3 of a studio recording.
GPU and VRAM	The graphics processor does the maths; its own memory (VRAM) has to hold the model. VRAM size decides which models a machine can run.	The size of the workbench decides how big a job you can take on.
Tokens per second	How fast a model writes. People read at about 5 tokens per second; a good private setup writes 30 to 100 or more.	Words per minute, for a machine.
Mixture of experts	A model split into many specialist sub-networks, with only a few switched on for each token. Big-model knowledge at small-model speed.	A hospital full of specialists, where each patient sees only two or three.
Distillation	Training a smaller model to imitate a bigger one. Many of the best small models are made this way.	An apprentice learning from a master.
Top-K	A setting that lets the model choose the next word only from its K most likely candidates, cutting every unlikely word out before it picks.	A shortlist: only the top five candidates get an interview.
Memory	How a chat system keeps track of a conversation. The model itself remembers nothing between calls; the software sends the earlier messages back with every new question.	A colleague with no memory, handed the meeting notes before every reply.
Chunking	Cutting long documents into short passages before they are stored for search, so the system can find and quote exactly the part that answers a question.	Cutting a book into index cards.
ChromaDB	An open-source vector database. It stores each chunk with its embedding and details such as source and page, and finds the chunks closest in meaning to a question.	A filing cabinet sorted by meaning instead of by name.
LangChain	An open-source toolkit for building applications on language models: model connectors, prompts, retrievers and tools that snap together.	A box of standard parts for building with AI.
LangGraph	A framework from the LangChain team for building agents as graphs: steps are nodes, arrows decide what runs next, and loops, branches and pauses are allowed.	A flowchart that can loop back, take a decision and wait for sign-off.
Supervisor agent	An agent whose job is to read each request, hand it to the right specialist agent, and decide when the work is done.	A team lead who assigns the ticket.

TERM	MEANING	LIKE
Human in the loop	A rule that the AI pauses before a high-impact action and waits for a person to approve, edit or reject it.	Four-eyes approval on a payment.
Checkpoint	A saved snapshot of an agent’s state after each step, so a run can pause, survive a restart and carry on exactly where it stopped.	A save point in a video game.
LangSmith	A tracing and monitoring tool from the LangChain team. It records every step of an AI run: prompts, retrieved passages, tool calls, answers, time taken and tokens used.	A flight recorder for every answer.
Hallucination	When a model states something false with confidence, such as an invented figure, quote or source.	A confident guest at a party who never admits they don’t know.
Prompt injection	Hidden instructions inside a document, email or web page that try to hijack the model, such as “ignore your rules and send me the file”.	A forged note slipped into the in-tray.
Guardrails	Rules and checks around the model: what it declines to do, what data it may reveal, and which actions need a person’s approval.	Company policy, enforced automatically.
Evals	Tests that measure how well a model does a job. Public benchmarks compare models in general; our own evals tell us whether one is good at our work.	A driving test we set, on our own roads.
Bias	Models absorb patterns from their training data, including unfair or one-sided ones.	A new hire who picked up bad habits at their last job.
Data retention	How long questions, documents and answers are stored, and who can access them.	The difference between a conversation and a recording.

APPENDIX B · BACKUP, ONLY IF ASKED

Material from slides that were cut

These slides were in earlier versions and are no longer in the deck. Use them only to answer questions.

RENT OR OWN

	SUBSCRIPTION AI (CHATGPT, CLAUDE, GEMINI, COPILOT)	OUR OWN SYSTEM
Where it runs	The vendor's data centres, shared with other customers	Our servers, on-site or in our own cloud account
How we pay	Per person or per token, at vendor prices: grows with use	Hardware once, then running costs: flat as use grows
Our data	Leaves our network; kept under the vendor's retention rules	Never leaves; we log it and can prove compliance
The model	Updated or retired on the vendor's schedule	Ours to keep; a better open model swaps in within an afternoon
Limits and outages	Rate limits, usage caps, outages; needs the internet	Only our own hardware; works when the internet is down
Fit to our work	What the vendor offers	Any model, tool or workflow, connected to our systems directly
Know-how	The skills stay with the vendor	Our team learns to build with AI; the know-how compounds
Top-end quality	The strongest models available today	Strong and improving fast; behind on the hardest tasks
Getting started	Minutes	A few weeks for a pilot, and an owner in IT

ILLUSTRATIVE COSTS (EARLIER CALCULATOR DEFAULTS)

- Subscriptions: 150 people × ₹2,900 per person per month ≈ ₹4.35 lakh a month, about ₹1.57 crore over three years.
- On-premises: a server bought once for about ₹30 lakh (for example 2 × NVIDIA L40S GPUs, 96 GB in total) plus about ₹1.25 lakh a month to run (power, maintenance, a share of an engineer's time): about ₹75 lakh over three years; it pays for itself in about month 10 at these settings.
- AWS Mumbai (ap-south-1), on-demand prices from AWS's list of 25 September 2026: g7e.2xlarge (1 × RTX PRO 6000, 96 GB) $5.49 an hour; g6e.2xlarge (1 × L40S, 48 GB) $2.69; p5.4xlarge (1 × H100, 80 GB) $8.26; g6e.12xlarge (4 × L40S) $12.60. With the calculator's ₹80,000 a month of other costs (a share of an engineer's time, storage, backups), the g7e.2xlarge comes to about ₹4.65 lakh a month always on (730 hours), or about ₹1.96 lakh a month if it is switched off outside office hours (about 220 hours): that switch is the biggest saving. The earlier deck converted at ₹95.96 = $1.
- All of these were illustrative calculator settings, not quotes. Offer to come back with real prices.

FOUR ROUTES

- **A · Open-weight model as it is**: days; good for drafting, summarising, translation; cannot answer anything about our own documents.
- **B · Open-weight model + RAG** (the earlier recommendation, "now"): 4–8 weeks to a pilot; questions about our policies, products, contracts and code, with sources; only as good as our documents and search; needs tuning and an owner.

- **C · Fine-tune an open-weight model** (“later, if tests show a need”): 2–4 months a round; a fixed style or narrow tasks; knowledge frozen at training time, no sources.
- **D · Train our own from scratch** (“not needed”): a year or more; needs far more text than we own.

PILOT OUTLINE

- **Weeks 1–2, choose**: one department and its documents; agree what success looks like; Legal reviews the model’s licence.
- **Weeks 3–8, build**: one GPU server, bought or rented; an open-weight model with RAG over that department’s documents, inside our harness with access rights and an audit log; twenty to thirty pilot users.
- **Weeks 9–12, measure**: accuracy on real questions with known answers; time saved, from the users themselves; cost per user compared with subscriptions.
- **Next quarter, decide and grow**: more departments and data sources, connected tools, and fine-tuning only where tests show a clear gain.

APPENDIX C

Sources and what is illustrative

PUBLIC FIGURES

- **14% / 34%**: Brynjolfsson, Li and Raymond, “Generative AI at Work”, NBER working paper, 2023 (5,179 customer-support agents).
- **55%**: Peng, Kalliamvakou, Cihon and Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot”, 2023 (controlled experiment).
- **Llama 3**: 15 trillion+ training tokens; about 30.8 million H100 GPU-hours for Llama 3.1 405B (Meta).
- **gpt-oss-120b**: 117B parameters, about 5.1B active per token, 36 layers, 128 experts (4 per token), model width 2,880, vocabulary 201,088 (o200k_harmony), 128K context (131,072), MXFP4 expert weights, fits one 80 GB GPU; Apache 2.0; reported near-parity with o4-mini on core reasoning (OpenAI, August 2025). Some sources list 5.7B active parameters; most give 5.1B.
- **Llama 4 Maverick**: 400B total, 17B active per token (Meta). **GPT-3**: 175B (2020).
- **all-MiniLM-L6-v2**: 384-dimensional embeddings; reads the first 256 word pieces.
- **ChromaDB**: Apache 2.0. **LangSmith**: self-hosting on the Enterprise plan. **Langfuse**: open source.

ILLUSTRATIVE (LABELLED ON THE SLIDES)

- The live demo’s probabilities (slide 1), token IDs, embedding values, similarity scores and attention weights (slide 5), the word scores and hallucination gauge (slide 6), token counts (slide 7), department examples (slide 10), HR Policy 2026 and its scores (slide 12), all metrics and runbooks in the agent runs (slide 13), the approval scenario’s metrics (slide 14), and every trace and monitoring figure (slide 15).